

**VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING**



**BÁO CÁO
ĐỒ ÁN CHUYÊN NGÀNH
HỌC KỲ 252 NĂM HỌC 2025-2026**

**ỨNG DỤNG AI CHO TRUY XUẤT VÀ TÓM TẮT
THÔNG TIN CUỘC HỌP NHIỀU NGƯỜI THAM
GIA**

CHUYÊN NGÀNH: KHOA HỌC MÁY TÍNH

HỘI ĐỒNG: Hội đồng 3L Đồ án chuyên ngành

SUPERVISOR(s): ThS. VÕ THANH HÙNG

REVIEWER: TS. Lê Thành Sách

ThS. Trần Huy

ThS. Vương Bá Thịnh

—o0o—

SINH VIÊN 1: PHẠM NGUYỄN ANH QUÂN - 2212811

SINH VIÊN 2: TRẦN THANH BÌNH - 2210335

Thành phố Hồ Chí Minh, THÁNG 5 2026

Giảng viên Hướng dẫn ký tên

_____ Date: _____

ThS. VÕ THANH HÙNG

Khoa Khoa học và Kỹ thuật Máy tính

Cam kết về tính xác thực

Chúng tôi xin cam kết rằng đồ án chuyên ngành này được thực hiện hoàn toàn bởi nhóm tác giả, dưới sự hướng dẫn của ThS. Võ Thanh Hùng tại Khoa Khoa học và Kỹ thuật Máy tính, Trường Đại học Bách khoa – Đại học Quốc gia Thành phố Hồ Chí Minh.

Tất cả các tài liệu, nguồn tham khảo và kết quả nghiên cứu từ bên ngoài được sử dụng trong đồ án đều đã được trích dẫn và ghi chú đầy đủ theo đúng quy định.

Trong trường hợp phát hiện bất kỳ hành vi sao chép hoặc vi phạm bản quyền nào, chúng tôi xin hoàn toàn chịu trách nhiệm. Trường Đại học Bách khoa – Đại học Quốc gia Thành phố Hồ Chí Minh không chịu trách nhiệm đối với các vi phạm bản quyền (nếu có) phát sinh trong quá trình thực hiện đồ án.

Thành phố Hồ Chí Minh, tháng 5 năm 2026

Tác giả

Lời cảm ơn

Chúng tôi xin chân thành cảm ơn ThS. Võ Thanh Hùng đã tận tình hướng dẫn và hỗ trợ trong suốt quá trình thực hiện đồ án. Những góp ý chuyên môn và phản biện mang tính xây dựng của thầy là cơ sở quan trọng giúp chúng tôi hoàn thiện hướng tiếp cận và hệ thống đề xuất.

Chúng tôi cũng xin cảm ơn các thầy cô, bạn bè và đồng nghiệp đã đóng góp nhiều ý kiến giá trị, giúp nhận diện rõ hơn các ưu điểm và hạn chế của những phương pháp hiện có, từ đó cải thiện chất lượng nghiên cứu.

Cuối cùng, chúng tôi xin gửi lời cảm ơn đến gia đình vì sự động viên và hỗ trợ liên tục trong suốt quá trình học tập và hoàn thành đồ án.

Tóm tắt Đồ án

Trong bối cảnh chuyển đổi số, việc khai thác tri thức từ dữ liệu âm thanh cuộc họp gặp nhiều trở ngại do đặc thù nhiều môi trường và sai sót từ quá trình nhận dạng tiếng nói tự động (ASR). Đồ án này tập trung nghiên cứu và hiện thực hóa một quy trình xử lý đầu cuối (*end-to-end pipeline*) nhằm tự động hóa việc truy xuất và tổng hợp thông tin từ các bản ghi âm hội thoại đa người nói.

Giải pháp đề xuất bao gồm một hệ thống tích hợp các mô hình Phân đoạn người nói (*Speaker Diarization*) và Nhận dạng tiếng nói (ASR) làm nền tảng số hóa dữ liệu. Trọng tâm kỹ thuật nằm ở kiến trúc RAG nâng cao (*Advanced RAG*) với cơ chế hiệu chỉnh ngữ nghĩa bằng Mô hình Ngôn ngữ Lớn (*LLM Correction*) và chiến lược truy xuất đa bước (*Multi-step Reasoning*) dựa trên phương pháp IRCot. Hệ thống còn triển khai cơ chế tóm tắt đa tầng (*Multi-level Summarization*) và lọc siêu dữ liệu (*Metadata Filtering*) để tối ưu hóa tính xác thực và khả năng truy vết thông tin.

Kết quả thực nghiệm trên tập dữ liệu AMI và tập dữ liệu chuyên ngành thực tế (BI-WASE) cho thấy kiến trúc Advanced RAG đạt chỉ số Hit Rate@5 lên tới 94%, vượt trội đáng kể so với phương pháp RAG truyền thống. Hệ thống thể hiện khả năng phục hồi tri thức tốt trước các sai sót ASR, đồng thời giảm mạnh kích thước dữ liệu lưu trữ với tỷ lệ nén (*Compression Ratio*) đạt khoảng 0.065 mà vẫn bảo toàn được các đơn vị thông tin cốt lõi. Nghiên cứu đã chứng minh tính hiệu quả của việc kết hợp giữa tìm kiếm ngữ nghĩa và suy luận logic trong việc quản lý tri thức doanh nghiệp từ nguồn dữ liệu âm thanh phi cấu trúc.

Contents

1	Giới Thiệu	1
1.1	Động Lực	1
1.2	Mục tiêu Đồ án	2
1.2.1	Mục tiêu tổng quát	2
1.2.2	Mục tiêu cụ thể	2
1.3	Phạm Vi & Giới Hạn	3
1.3.1	Phạm vi bài toán	3
1.3.2	Giới hạn của bài toán	4
1.4	Cấu Trúc Báo Cáo	5
2	Công Nghệ Liên Quan	7
2.1	Những Thư Viện, Công Cụ Hiện Có	7
2.1.1	Các hệ thống và nền tảng có sẵn	8
3	Cơ Sở Lý Thuyết	10
3.1	Phát Hiện Hoạt Động Giọng Nói - VAD	10
3.1.1	Định nghĩa VAD	10
3.1.2	Ảnh hưởng của VAD	10
3.2	Biểu Diễn Đặc Trưng - Speech Embeddings	11
3.2.1	Mục tiêu	11
3.3	Phân Cụm Người Nói - Speaker Clustering	12
3.3.1	Mục tiêu	12
3.4	Chuyển đổi giọng nói thành văn bản - ASR	13
3.4.1	Các kiến trúc ASR	13

3.5	Tìm kiếm ngữ nghĩa (Semantic Search)	14
3.5.1	Định nghĩa	14
3.5.2	Vai trò và Tầm quan trọng	14
3.6	Xếp hạng văn bản (Re-ranking)	15
3.6.1	Định nghĩa	15
3.6.2	Vai trò trong hệ thống truy xuất	15
3.7	Cơ sở tri thức và Cơ sở dữ liệu Vector	16
3.7.1	Cơ sở tri thức (Knowledge Base)	16
3.7.2	Cơ sở dữ liệu Vector (Vector Database)	17
3.7.3	Mô hình biểu diễn văn bản (Text Embeddings)	17
3.7.4	Kỹ thuật chỉ mục (Indexing)	17
3.8	Cơ sở lý thuyết về RAG	18
3.8.1	Tổng quan về Mô hình Ngôn ngữ Lớn (LLM)	18
3.8.2	Hạn chế của LLM: Vấn đề ảo giác và Tri thức tĩnh	18
3.8.3	Giải pháp RAG: Neo giữ thông tin và Kiến trúc lai	19
3.8.4	Ứng dụng thực tiễn và Bảo mật dữ liệu	20
3.9	Truy xuất và Suy luận đa bước (Multi-step Retrieval and Reasoning) . .	21
3.9.1	Thách thức của các truy vấn đa bước (Multi-hop queries)	21
3.9.2	Kỹ thuật Chuỗi suy luận (Chain-of-Thought - CoT)	21
3.9.3	Phương pháp Lồng ghép Truy xuất và Suy luận (IRCoT)	21
3.9.4	Khả năng ứng dụng trong hệ thống RAG hội thoại	22
3.10	Hệ thống tóm tắt đa tầng hỗ trợ truy xuất và tìm kiếm thông tin	22
3.10.1	Transcript Layer	24
3.10.2	Participant Layer	24
3.10.3	Topic Layer	25
3.10.4	Meeting Layer	26
3.10.5	Đặc điểm của hệ thống tóm tắt đa tầng	26
3.11	Số liệu đánh giá	27
3.11.1	Các chỉ số đánh giá (Evaluation Metrics)	27
4	Thiết Kế & Hiện Thực Hệ Thống	33
4.1	Tổng quan kiến trúc hệ thống	33

4.1.1	Mô hình kiến trúc tổng thể	33
4.1.2	Luồng dữ liệu trong hệ thống	34
4.1.3	Các nguyên lý thiết kế cốt lõi	34
4.2	Mô hình Xử lý tiếng nói (Speech Processing Module)	35
4.2.1	Định danh người nói	36
4.2.2	Mô hình Tự động Nhận diện Giọng nói	37
4.3	Mô hình Truy xuất và Tạo sinh tri thức (RAG Module)	38
4.3.1	Xây dựng và Đánh chỉ mục dữ liệu (Indexing)	41
4.3.2	Quy trình Truy xuất đa tầng (Multi-stage Retrieval)	44
4.3.3	Quy trình Truy xuất và Suy luận đa bước (Multi-step Retrieval and Reasoning)	48
4.3.4	Tích hợp Mô hình ngôn ngữ và Tạo sinh (Generation)	49
5	Đánh Giá	53
5.1	Đánh giá Hệ thống	53
5.1.1	Dữ liệu đánh giá (Evaluation Dataset)	53
5.1.2	Đánh giá Định danh người nói	54
5.1.3	Đánh giá tổng quan pipeline chuyển đổi âm thanh thành văn bản	57
5.1.4	Đánh giá tóm tắt đa tầng	59
5.1.5	So sánh hiệu năng truy xuất (Advanced RAG và Naive RAG) .	61
5.1.6	Đánh giá hiệu quả của siêu dữ liệu (Metadata Filtering)	63
5.1.7	Đánh giá khả năng phục hồi tri thức và chịu lỗi của hệ thống RAG	65
6	Kết Luận	67
6.1	Kết luận	67
6.1.1	Những hạn chế	68
6.1.2	Định hướng phát triển	69

List of Tables

3.1	So sánh hiệu quả giữa LLM thuần túy và RAG trong doanh nghiệp . . .	20
5.1	So sánh ECAPA-TDNN và TitaNet trên các chỉ số diarization	55
5.2	Kết quả đánh giá pipeline chuyển đổi âm thanh thành văn bản	58
5.3	Kết quả đánh giá hệ thống tóm tắt đa tầng	60

List of Figures

3.1	So sánh kiến trúc Bi-Encoder và Cross-Encoder	16
3.2	Kiến trúc hệ thống tóm tắt đa tầng hỗ trợ truy xuất và tìm kiếm thông tin	23
4.1	Sơ đồ luồng xử lý dữ liệu tích hợp chức năng Tóm tắt trong mô-đun RAG	40
4.2	Sơ đồ quy trình truy xuất hai giai đoạn (Retrieval & Re-ranking)	44
5.1	So sánh hiệu năng truy xuất (Hit Rate @K) giữa phương pháp phân mảnh nguyên câu (Naive RAG) và hệ thống đề xuất với chiến lược Parent Doc- ument Retrieval (Advanced RAG)	62

Chapter 1

Giới Thiệu

1.1 Động Lực

Trong bối cảnh chuyển đổi số, các cuộc họp trực tuyến và trực tiếp đóng vai trò thiết yếu trong hoạt động vận hành của doanh nghiệp, tạo ra một khối lượng khổng lồ dữ liệu âm thanh phi cấu trúc. Tuy nhiên, việc khai thác giá trị tri thức từ nguồn dữ liệu này gặp nhiều trở ngại do đặc thù phức tạp của hội thoại tự nhiên: sự luân phiên người nói liên tục, hiện tượng chồng lấn tín hiệu (overlapping speech), và các sai số không thể tránh khỏi trong quá trình nhận dạng tiếng nói tự động (ASR). Các phương pháp tra cứu thủ công hoặc tìm kiếm từ khóa truyền thống thường tốn kém thời gian, độ chính xác thấp và khó mở rộng.

Xuất phát từ thực tế đó, đề án đặt ra vấn đề xây dựng một hệ thống tự động hóa có khả năng chuyển đổi dữ liệu âm thanh hội họp thành tri thức có cấu trúc. Yêu cầu cốt lõi không chỉ dừng lại ở việc trích xuất nội dung văn bản, mà còn phải bảo toàn được ngữ cảnh hội thoại (định danh người nói, mốc thời gian) và đảm bảo tính xác thực thông qua khả năng truy vết ngược về dữ liệu gốc.

Mục tiêu chính của đề án là thiết kế và hiện thực hóa một quy trình xử lý đầu cuối (end-to-end pipeline) cho bài toán truy xuất và tổng hợp thông tin cuộc họp. Thay vì chỉ tập trung tối ưu hóa cục bộ từng thành phần riêng lẻ, đề án hướng tới việc lựa chọn và tích hợp các mô hình có độ ổn định cao, đảm bảo sự đồng bộ cho toàn hệ thống và giảm thiểu tối đa sự lan truyền lỗi giữa các giai đoạn xử lý từ âm thanh đến văn bản và cuối cùng là truy xuất ngữ nghĩa.

1.2 Mục tiêu Đồ án

1.2.1 Mục tiêu tổng quát

Mục tiêu chính của đồ án là nghiên cứu, thiết kế và hiện thực hóa một quy trình xử lý đầu cuối (end-to-end pipeline) nhằm tự động hóa việc khai thác tri thức từ dữ liệu âm thanh hội họp. Hệ thống hướng tới khả năng vận hành ổn định trên các dữ liệu thực tế có độ phức tạp cao (đa người nói, nhiều môi trường), giúp chuyển đổi tài nguyên âm thanh phi cấu trúc thành thông tin có giá trị sử dụng.

1.2.2 Mục tiêu cụ thể

Để đạt được mục tiêu tổng quát nêu trên, đồ án tập trung giải quyết các nhiệm vụ cụ thể sau:

Thiết lập pipeline tiền xử lý âm thanh: Sử dụng các mô hình tiền huấn luyện (pre-trained models) hiện hành cho bài toán Phân đoạn người nói (Speaker Diarization) và Nhận dạng tiếng nói tự động (ASR). Mục tiêu là thiết lập luồng dữ liệu tự động để trích xuất văn bản và siêu dữ liệu (người nói, thời gian) nhằm tạo đầu vào thực tế cho hệ thống.

Phát triển kiến trúc RAG chịu lỗi: Xây dựng cơ sở dữ liệu vector và hệ thống truy xuất thông tin có khả năng hoạt động hiệu quả trên tập dữ liệu thực tế mang nhiễu (lỗi nhận diện từ vựng, sai lệch ranh giới người nói). Trọng tâm đặt vào việc ứng dụng phương pháp tìm kiếm ngữ nghĩa (Semantic Search) và năng lực của LLM để hậu xử lý, chuẩn hóa và suy luận đúng thông tin thay vì phụ thuộc vào một bản ghi (transcript) hoàn hảo.

Xử lý truy vấn phức tạp, đa bước: Nâng cấp cơ chế RAG để giải quyết các câu hỏi đòi hỏi khả năng suy luận qua nhiều bước hoặc kết hợp thông tin từ nhiều nguồn. Tích hợp linh hoạt kết quả tìm kiếm vector với cơ chế lọc theo siêu dữ liệu (Metadata Filtering) nhằm định vị chính xác thông tin bị phân tán trong các bản ghi hội thoại dài.

Xây dựng chiến lược tóm tắt hội thoại đa tầng (Multi-level Summarization): Ứng dụng Mô hình Ngôn ngữ Lớn (LLM) để giải quyết bài toán ngữ cảnh dài (Long-context), phát triển hệ thống tóm tắt theo ba cấp độ: tóm tắt tổng quan toàn bộ cuộc họp, tóm tắt chi tiết theo từng chủ đề thảo luận và tổng hợp các phát biểu trọng tâm của từng cá nhân tham gia.

Xây dựng phương pháp đánh giá hệ thống: Thực nghiệm và đo lường hiệu năng của hệ thống trên các kịch bản thực tế. Tập trung đánh giá khả năng ”tự sửa lỗi” và phục hồi ngữ nghĩa của hệ thống RAG trước các sai số từ ASR, mức độ đóng góp của siêu dữ liệu vào độ chính xác truy xuất, cũng như kiểm chứng tính xác thực (faithfulness) của các mô-đun sinh văn bản.

1.3 Phạm Vi & Giới Hạn

1.3.1 Phạm vi bài toán

Trong khuôn khổ đề án tốt nghiệp, hệ thống tập trung giải quyết bài toán truy xuất và tổng hợp thông tin từ dữ liệu âm thanh cuộc họp nhiều người. Thay vì đi sâu vào nghiên cứu các thuật toán xử lý tín hiệu số, đề án giới hạn phạm vi nghiên cứu trọng tâm vào tầng xử lý ngôn ngữ tự nhiên (NLP) với các định hướng cụ thể sau:

Dữ liệu đầu vào và Ngôn ngữ: Đầu vào của hệ thống là các bản ghi âm hội thoại (chủ yếu bằng tiếng Anh và hỗ trợ tiếng Việt) có thời lượng dài, được thu thập trong môi trường thực tế (có chứa nhiều nền và hiện tượng nói chồng lấp). Hệ thống chấp nhận và hướng tới việc xử lý các bản ghi văn bản (transcript) đầu ra từ mô hình nhận dạng có chứa sai số.

Tiền xử lý âm thanh: Đề án kế thừa và sử dụng các mô hình tiền huấn luyện (pre-trained models) mã nguồn mở cho các tác vụ Phát hiện giọng nói (VAD), Định danh người nói (Speaker Diarization) và Nhận dạng tiếng nói (ASR) như những mô-đun độc lập. Hệ thống không thực hiện huấn luyện lại (retrain) hay can thiệp sâu vào kiến trúc của các mô hình xử lý tín hiệu này.

Phát triển hệ thống End-to-End RAG: Xây dựng ứng dụng hoàn chỉnh cho phép

người dùng truy vấn thông tin bằng ngôn ngữ tự nhiên. Phạm vi chức năng cốt lõi bao gồm: thực hiện tóm tắt hội thoại đa tầng và trích xuất dẫn chứng chính xác (bao gồm mốc thời gian và định danh người phát biểu) từ kho dữ liệu hội thoại.

Tối ưu hóa Truy xuất thông tin (IR) trên văn bản nhiều: Trọng tâm kỹ thuật nằm ở việc nghiên cứu các phương pháp hậu xử lý dữ liệu. Điển hình là sử dụng mô hình ngôn ngữ lớn (LLM) để hiệu chỉnh lỗi (LLM correction), tối ưu hóa luồng truy xuất thông tin (IR Optimization) và tích hợp sâu siêu dữ liệu (Speaker, Timestamp) để tăng cường khả năng biểu diễn trong không gian vector.

Đánh giá hiệu năng chuyên sâu: Đồ án tập trung đo lường sức mạnh của hệ thống thông qua các chỉ số cốt lõi của kiến trúc RAG như Hit Rate, MRR, Faithfulness và Answer Correctness trên các tập dữ liệu có độ khó thực tế, thay vì chỉ đánh giá các chỉ số nhận dạng âm thanh đơn thuần.

Môi trường triển khai: Hệ thống được thiết kế theo hướng mô-đun, sử dụng các mô hình học sâu quy mô vừa triển khai trên GPU phổ thông. Do thành phần LLM cốt lõi được tích hợp thông qua API, hệ thống hoạt động theo cơ chế bán trực tuyến (semi-online) và yêu cầu kết nối mạng ổn định trong quá trình vận hành.

1.3.2 Giới hạn của bài toán

Bên cạnh những mục tiêu đạt được, đồ án xác định rõ các ranh giới nghiên cứu dựa trên các kết quả thực nghiệm và định hướng phát triển hệ thống như sau:

Tính kế thừa và đóng gói từ giai đoạn nghiên cứu tiền đề: Hệ thống kế thừa các cấu hình tối ưu về Phân đoạn người nói và Nhận dạng tiếng nói (ASR) đã được kiểm thử và đánh giá chi tiết trong giai đoạn nghiên cứu trước đó. Đồ án thực hiện đóng gói các kết quả này thành các mô-đun tiền xử lý ổn định để làm nền tảng dữ liệu cho hệ thống tổng thể, thay vì tiếp tục tập trung tối ưu hóa cục bộ ở tầng tín hiệu vốn đã tiệm cận ngưỡng giới hạn về hiệu năng trước các dữ liệu nhiễu thực tế.

Tập trung vào khả năng phục hồi tri thức: Nhận diện được các sai số không thể tránh khỏi của tầng nhận dạng âm thanh trong môi trường thực tế, đồ án tập trung nguồn lực vào việc xây dựng cơ chế tự sửa lỗi ở tầng hậu xử lý. Trọng tâm nghiên

cứu được đặt vào việc ứng dụng LLM để hiệu chỉnh ngữ nghĩa (LLM Correction) và tối ưu hóa quy trình RAG nhằm đảm bảo tính xác thực của thông tin ngay cả khi dữ liệu đầu vào có sai lệch.

Tác động của sai số tích lũy và ngưỡng phục hồi tri thức: Chất lượng của toàn bộ pipeline phụ thuộc đáng kể vào sai số tích lũy từ các bước xử lý tiếng nói, đặc biệt là lỗi bỏ sót tiếng nói (*Miss*) và lỗi nhầm lẫn người nói (*Confusion*). Mặc dù kiến trúc RAG được thiết kế để chịu lỗi, nhưng nếu các sai số tích lũy này làm mất hoàn toàn các thực thể định danh cốt lõi hoặc biến dạng sâu sắc các con số then chốt, hệ thống vẫn tồn tại rủi ro trong việc trích xuất và tổng hợp thông tin chính xác tuyệt đối.

Giới hạn về tính đa dạng của hội thoại tự nhiên: Đồ án tập trung giải quyết các đặc thù chung của hội thoại trong môi trường hội họp và hạ tầng. Tuy nhiên, khả năng thích nghi với các biến thể ngôn ngữ cực đoan như từ lóng quá đặc thù, phương ngữ nặng hoặc các đoạn hội thoại có mật độ chồng lấn lời nói (*overlapping speech*) quá dày đặc vẫn là một thách thức chưa được giải quyết triệt để trong phạm vi này.

Chế độ vận hành và độ trễ hệ thống: Do quy trình thực thi bao gồm nhiều bước hậu xử lý logic phức tạp để đảm bảo độ chính xác (như *Query Rewriting*, *Re-ranking* và *Summarization*), tốc độ phản hồi của hệ thống sẽ có độ trễ nhất định. Vì vậy, hệ thống chưa tối ưu cho các kịch bản thời gian thực mà chủ yếu được thiết kế cho xử lý ngoại tuyến (*offline processing*). Ngoài ra, việc sử dụng các mô hình ngôn ngữ lớn thông qua API khiến hệ thống chịu sự chi phối về tính ổn định và chính sách từ nhà cung cấp dịch vụ bên thứ ba.

1.4 Cấu Trúc Báo Cáo

Báo cáo được tổ chức thành các chương với nội dung và vai trò rõ ràng nhằm trình bày tuần tự từ bối cảnh bài toán đến thiết kế, triển khai và đánh giá hệ thống:

Chương 1 – Giới thiệu: Trình bày bối cảnh bài toán, động lực thực hiện, mục tiêu của đồ án, phạm vi và cấu trúc tổng thể của báo cáo.

Chương 2 – Các công trình liên quan: Tổng quan các hướng tiếp cận và hệ thống tiêu biểu trong lĩnh vực xử lý hội thoại nhiều người, nhận dạng tiếng nói và truy xuất thông tin, làm cơ sở định vị hướng tiếp cận của đồ án.

Chương 3 – Cơ sở lý thuyết: Trình bày các khái niệm và nền tảng kỹ thuật cần thiết như phát hiện hoạt động giọng nói, biểu diễn embedding, phân cụm, nhận dạng tiếng nói và truy xuất văn bản, phục vụ cho việc hiểu các chương sau.

Chương 4 – Thiết kế và hiện thực hệ thống: Mô tả kiến trúc tổng thể của hệ thống, luồng dữ liệu và thiết kế chi tiết các mô-đun chính, bao gồm mô-đun xử lý tiếng nói và mô-đun truy xuất và tổng hợp thông tin.

Chương 5 – Đánh giá và thảo luận: Trình bày các kịch bản đánh giá, tiêu chí đo lường và kết quả thực nghiệm nhằm so sánh các cấu hình hệ thống và phân tích các yếu tố ảnh hưởng đến hiệu năng.

Chương 6 – Kết luận: Tổng kết các kết quả đạt được, nêu hạn chế của hệ thống và định hướng phát triển trong tương lai.

Chapter 2

Công Nghệ Liên Quan

2.1 Những Thư Viện, Công Cụ Hiện Có

Hệ thống được xây dựng dựa trên các thư viện và công cụ mã nguồn mở phổ biến trong lĩnh vực xử lý tiếng nói và truy xuất thông tin. Các thành phần này có thể được sử dụng độc lập dưới dạng mô-đun hoặc kết hợp thành pipeline hoàn chỉnh:

Thư viện xử lý tiếng nói: Các thư viện hỗ trợ phát hiện hoạt động giọng nói, trích xuất embedding và nhận dạng tiếng nói, cho phép xây dựng pipeline diarization và ASR theo từng bước.

Mô hình end-to-end: Một số mô hình cung cấp khả năng xử lý trọn gói (ví dụ ASR end-to-end), được sử dụng như các khối chức năng thay thế nhằm so sánh và đánh giá tính linh hoạt của hệ thống.

Thư viện học máy và học sâu: Các framework như PyTorch được sử dụng để tải và suy luận mô hình, đồng thời hỗ trợ thao tác tensor và tăng tốc bằng GPU.

Công cụ truy xuất và lưu trữ vector: Các thư viện phục vụ việc xây dựng chỉ mục vector và truy vấn ngữ nghĩa, đóng vai trò trung tâm trong mô-đun RAG.

API mô hình ngôn ngữ lớn: LLM được tích hợp thông qua API, cho phép thực hiện các tác vụ tổng hợp, trả lời câu hỏi và tái cấu trúc thông tin dựa trên ngữ cảnh truy xuất.

2.1.1 Các hệ thống và nền tảng có sẵn

Hiện nay đã tồn tại nhiều hệ thống, framework và dịch vụ có khả năng giải quyết toàn bộ hoặc một phần bài toán truy xuất và tổng hợp thông tin từ cuộc họp nhiều người. Các giải pháp này có thể được phân loại theo phạm vi xử lý như sau:

Hệ thống xử lý tiếng nói (Speech Processing):

- *Whisper* (OpenAI): mô hình ASR end-to-end, hỗ trợ đa ngôn ngữ, có độ ổn định cao trong môi trường hội họp nhưng không tích hợp trực tiếp speaker diarization.
- *pyannote.audio*: framework chuyên sâu cho các tác vụ phân đoạn tiếng nói và định danh người nói, được sử dụng rộng rãi trong nghiên cứu diarization, nhưng yêu cầu tích hợp với ASR và các thành phần phía sau.
- *NVIDIA NeMo*: nền tảng cung cấp các mô hình ASR, diarization và speech processing theo hướng mô-đun, phù hợp cho xây dựng pipeline nhưng đòi hỏi tài nguyên phần cứng lớn.
- *SpeechBrain*: thư viện nghiên cứu hỗ trợ ASR, diarization và embedding tiếng nói, linh hoạt nhưng chưa tối ưu cho triển khai quy mô lớn.

Hệ thống truy xuất và tổng hợp thông tin (Retrieval & Summarization):

- *LangChain*: framework xây dựng pipeline RAG, hỗ trợ tích hợp LLM, embedding và vector database, phù hợp cho truy xuất ngữ nghĩa từ văn bản hội thoại.
- *LlamaIndex*: hệ thống tổ chức và truy xuất dữ liệu văn bản quy mô lớn, hỗ trợ nhiều chiến lược indexing và retrieval, nhưng không xử lý trực tiếp dữ liệu âm thanh.
- *Haystack*: framework RAG hướng sản phẩm, hỗ trợ retrieval và question answering, yêu cầu dữ liệu đầu vào đã ở dạng văn bản.

Hệ thống end-to-end thương mại:

- *Otter.ai, Fireflies.ai, Zoom AI Companion*: cung cấp giải pháp hoàn chỉnh cho ghi âm, nhận dạng, tóm tắt và truy vấn nội dung cuộc họp. Các hệ thống này dễ sử dụng nhưng hạn chế khả năng tùy chỉnh, khó kiểm soát pipeline xử lý và không phù hợp cho phân tích thuật toán chi tiết.

Mặc dù các hệ thống trên có thể đáp ứng tốt nhu cầu ứng dụng thực tế, chúng hoặc chỉ giải quyết một phần bài toán, hoặc hoạt động như hộp đen ở mức end-to-end. Trong khuôn khổ đề án, việc xây dựng hệ thống theo pipeline mô-đun dựa trên các framework như Whisper, pyannote hoặc NeMo cho phép kiểm soát chi tiết từng bước xử lý, phân tích lỗi và đánh giá tác động của từng thành phần tới chất lượng truy xuất và tổng hợp thông tin.

Chapter 3

Cơ Sở Lý Thuyết

Chương này trình bày nhưng kiến thức sơ bộ được sử dụng trong hệ thống.

3.1 Phát Hiện Hoạt Động Giọng Nói - VAD

3.1.1 Định nghĩa VAD

Phát hiện Hoạt động Giọng Nói (VAD) là kỹ thuật nhằm xác định các khoảng thời gian trong dữ liệu âm thanh có chứa giọng nói của con người, phân biệt với thời gian im lặng, tiếng ồn nền hoặc âm thanh phi ngôn ngữ. Đây là bước tiền xử lý quan trọng giúp giảm khối lượng công việc ở các bước sau.

3.1.2 Ảnh hưởng của VAD

VAD ảnh hưởng trực tiếp đến nhiều phần của xử lý âm thanh, VAD xấu có thể:

Cắt mất lời nói, làm giảm độ chính xác của nhận dạng giọng nói.

Nhiều với tiếng ồn, khoảng lặng, làm cản trở việc trích xuất đặc trưng và phân cụm người nói.

Gia tăng chi phí tính toán không cần thiết.

3.2 Biểu Diễn Đặc Trưng - Speech Embeddings

3.2.1 Mục tiêu

Khi xử lý cuộc hội thoại, trích xuất đặc trưng được sử dụng để phân biệt giọng nói giữa nhiều người. Các kiến trúc được xem xét trong phần này là ECAPA-TDNN, TitaNet, WavLM; mỗi kiến trúc có ưu và nhược điểm riêng, và ảnh hưởng đến khả năng phân biệt và chất lượng embedding trong bài toán phân cụm.

Trong bối cảnh sử dụng trích xuất đặc trưng để tiến hành phân cụm, vai trò của mô hình Embeddings tập trung vào khoảng cách giữa các nhóm trong không gian đặc trưng.

ECAPA-TDNN là kiến trúc mở rộng của kiến trúc Time Delay Neural Network (TDNN), với cơ chế attention và multi-scale temporal context.

TitaNet là một kiến trúc neural sử dụng các convolution 1-chiều kết hợp với channel attention và global context pooling, được thiết kế để trích xuất biểu diễn speaker (t-vector) với khả năng mở rộng về kích thước mô hình.

Vì Embedding là đầu vào cho bài toán phân cụm người nói, sự khác biệt giữa các kiến trúc trên sẽ không được đánh giá độc lập mà thể hiện thông qua chất lượng phân cụm trong không gian đặc trưng, được phản ánh bởi chỉ số đánh giá tổng hợp như Diarization Error Rate (DER).

Hai mô hình trên có khác biệt về thiết kế kiến trúc và quy mô, nghiên cứu trước đây cho thấy không tồn tại khác biệt thực nghiệm rõ ràng về chất lượng embeddings giữa hai mô hình [1]

Khác với ECAPA-TDNN và TitaNet, mô hình WavLM không được sử dụng trong đề án này như một speaker embedding chính phục vụ trực tiếp cho bài toán phân cụm người nói. WavLM là một mô hình biểu diễn speech dựa trên self-supervised learning với kiến trúc Transformer, được huấn luyện nhằm học các biểu diễn ngữ cảnh tổng quát cho nhiều tác vụ xử lý tiếng nói khác nhau.

Các nghiên cứu trước đây cho thấy embedding trích xuất từ WavLM mang tính ngữ cảnh cao và phù hợp cho các tác vụ ở mức phân đoạn ngắn, tuy nhiên các biểu diễn này không được tối ưu riêng cho việc phân biệt danh tính người nói và do đó thường không

thể hiện ưu thế rõ rệt trong các bài toán speaker clustering khi so sánh với các kiến trúc embedding chuyên dụng như ECAPA-TDNN hoặc TitaNet. [1]

3.3 Phân Cụm Người Nói - Speaker Clustering

3.3.1 Mục tiêu

Trong bài toán phân cụm người nói, mục tiêu của clustering là nhóm đoạn biểu diễn đặc trưng giọng nói, sao cho các đoạn cùng thuộc một người nói được gán vào cùng một cụm trong khi số lượng người nói thường không được biết trước. Do chất lượng phân cụm phụ thuộc vào chất lượng biểu diễn đặc trưng, việc đánh giá trong đề án này sẽ là một phần thuộc pipeline tổng thể.

Hai hướng tiếp cận clustering phổ biến được sử dụng trong speaker diarization là Agglomerative Hierarchical Clustering (AHC) và Spectral Clustering.

AHC là phương pháp phân cụm bottom-up, trong đó mỗi đoạn giọng nói ban đầu được xem như một cụm riêng biệt, và các cụm được hợp nhất dần dựa trên độ tương đồng trong không gian embedding. Quá trình hợp nhất tiếp diễn cho tới khi đạt một ngưỡng dừng nhất định, thường được xác định thông qua một ngưỡng khoảng cách hoặc độ tương đồng.

Spectral Clustering phân cụm thông qua biểu diễn đồ thị, trong đó các đoạn giọng nói được xem như các đỉnh và độ tương đồng giữa các embedding được mã hóa thành các cạnh của đồ thị. Bằng cách chiếu dữ liệu sang không gian riêng (eigenspace) có chiều thấp hơn, spectral clustering cho phép tách các cụm dựa trên cấu trúc toàn cục của dữ liệu thay vì chỉ dựa trên khoảng cách cục bộ.

Trong speaker diarization, spectral clustering thường cho kết quả ổn định hơn trong các trường hợp embedding bị nhiễu hoặc phân bố không đồng đều. Tuy nhiên, phương pháp này yêu cầu xây dựng và xử lý ma trận tương đồng, dẫn đến chi phí tính toán cao hơn và phụ thuộc nhiều vào chất lượng hàm đo độ tương đồng.

Spectral clustering đã được chứng minh hiệu quả trong một số trường hợp cụ thể, chẳng hạn như xử lý overlap speaker hoặc dữ liệu có phân bố phức tạp [11]. Tuy nhiên, không có bằng chứng rõ ràng rằng spectral clustering luôn cho DER thấp hơn AHC, và hiệu

quả của spectral clustering phụ thuộc nhiều vào cách xây dựng ma trận tương đồng và chất lượng embedding đầu vào.

3.4 Chuyển đổi giọng nói thành văn bản - ASR

Trong hệ thống xử lý hội thoại, Automatic Speech Recognition (ASR) đóng vai trò chuyển đổi tín hiệu giọng nói thành văn bản nhằm phục vụ các tác vụ truy xuất, tìm kiếm và phân tích nội dung hội thoại.

Hai kiến trúc ASR phổ biến là Whisper và Wav2Vec2-CTC được sử dụng trong các ngữ cảnh khác nhau của hệ thống.

3.4.1 Các kiến trúc ASR

Một hệ thống ASR hiện đại thường bao gồm ba thành phần chính: mô hình âm học để ánh xạ tín hiệu âm thanh sang các biểu diễn ngôn ngữ trung gian, mô hình ngôn ngữ để mô hình hóa xác suất chuỗi từ, và bước giải mã để suy ra chuỗi văn bản có xác suất cao nhất. Trong các kiến trúc ASR gần đây, các thành phần này có thể được huấn luyện tách rời hoặc tích hợp trong một mô hình end-to-end thống nhất.

Whisper là một mô hình ASR end-to-end dựa trên kiến trúc Transformer, được huấn luyện trên tập dữ liệu đa ngôn ngữ quy mô lớn theo hướng weakly supervised learning. Mô hình tích hợp chặt chẽ giữa acoustic modeling và language modeling trong cùng một kiến trúc, cho phép xử lý linh hoạt nhiều ngôn ngữ và điều kiện âm thanh khác nhau.

Trong hệ thống, Whisper được sử dụng như một mô hình ASR tổng quát cho dữ liệu tiếng Anh, tận dụng khả năng thích nghi tốt với nhiều và sự đa dạng về domain. Tuy nhiên, trong phạm vi đề án, Whisper không được đặt trong bối cảnh so sánh hiệu năng trực tiếp với các mô hình ASR khác.

Wav2Vec2-CTC là một kiến trúc ASR dựa trên self-supervised learning, trong đó mô hình Wav2Vec2 được sử dụng để trích xuất biểu diễn âm thanh, kết hợp với hàm mất mát Connectionist Temporal Classification (CTC) để thực hiện giải mã chuỗi ký tự. Khác với các mô hình end-to-end tích hợp, kiến trúc này cho phép

tách biệt rõ ràng giữa quá trình học biểu diễn âm thanh và bước suy diễn chuỗi văn bản.

3.5 Tìm kiếm ngữ nghĩa (Semantic Search)

3.5.1 Định nghĩa

Tìm kiếm ngữ nghĩa (*Semantic Search*) là phương pháp truy xuất thông tin tập trung vào ý định của người dùng và ngữ cảnh của truy vấn, thay vì chỉ dựa trên sự trùng khớp từ khóa (*Lexical Search*) như các công cụ tìm kiếm truyền thống [2]. Cơ sở toán học của phương pháp này là Mô hình không gian vector (*Vector Space Model*), trong đó văn bản được chuyển đổi thành các vector số học (*embeddings*) nằm trong một không gian nhiều chiều [14].

Trong không gian này, vị trí của mỗi vector được xác định bởi các đặc trưng ngữ nghĩa của văn bản mà nó đại diện. Nguyên lý cốt lõi là các đoạn văn bản mang ý nghĩa tương đồng sẽ nằm gần nhau về mặt hình học, bất kể chúng có sử dụng chung từ vựng hay không. Quá trình này thường được thực hiện bởi các mô hình học sâu (*Deep Learning*) hoặc các mô hình ngôn ngữ lớn - *Large Language Model (LLM)*, cho phép máy tính “hiểu” được ngôn ngữ tự nhiên ở mức độ trừu tượng.

3.5.2 Vai trò và Tầm quan trọng

Trong bối cảnh Natural Language Processing (NLP) hiện đại, tìm kiếm ngữ nghĩa đóng vai trò then chốt trong việc giải quyết vấn đề “bất đồng ngôn ngữ” (*vocabulary mismatch*). Đây là hiện tượng thường gặp khi người dùng sử dụng các từ ngữ khác biệt (từ đồng nghĩa, từ lóng) so với các từ ngữ chính xác được lưu trữ trong tài liệu, khiến các phương pháp truyền thống như BM25 gặp khó khăn [5].

Việc áp dụng tìm kiếm ngữ nghĩa mang lại ảnh hưởng tích cực đáng kể đến hiệu suất của hệ thống truy xuất. Phương pháp này cho phép hệ thống vượt qua rào cản về mặt câu chữ để nắm bắt chính xác nhu cầu tin tức thông qua các biểu diễn vector ngữ cảnh chất lượng cao [12]. Ví dụ, một truy vấn về “hạn chót hoàn thành” có thể tìm thấy kết quả chứa từ “deadline” mà không cần khai báo từ điển đồng nghĩa thủ công.

3.6 Xếp hạng văn bản (Re-ranking)

3.6.1 Định nghĩa

Xếp hạng văn bản (*Re-ranking*) là kỹ thuật tinh chỉnh kết quả tìm kiếm, thường được áp dụng trong giai đoạn hai của các hệ thống truy xuất thông tin nhiều tầng. Trong kiến trúc này, sau khi một tập hợp các tài liệu ứng viên được truy xuất sơ bộ bởi các thuật toán tìm kiếm tốc độ cao (như tìm kiếm vector hoặc từ khóa), mô hình Re-ranking sẽ được sử dụng để đánh giá lại mức độ phù hợp của từng tài liệu đối với câu truy vấn một cách chi tiết hơn.

Về mặt kỹ thuật, các mô hình xếp hạng hiện đại thường dựa trên kiến trúc **Cross-Encoder**. Khác với **Bi-Encoder** (vốn mã hóa truy vấn và văn bản thành hai vector độc lập), Cross-Encoder tiếp nhận đầu vào là cặp dữ liệu (truy vấn, văn bản) đồng thời. Mô hình sử dụng cơ chế sự chú ý (*Self-Attention*) trong mạng nơ-ron Transformer để phân tích mối quan hệ tương tác ngữ nghĩa giữa từng từ của câu truy vấn với toàn bộ nội dung văn bản, từ đó xuất ra một điểm số tương đồng (*relevance score*) duy nhất [9].

3.6.2 Vai trò trong hệ thống truy xuất

Trong bài toán truy xuất thông tin, luôn tồn tại sự đánh đổi giữa độ phủ (*Recall*) và độ chính xác (*Precision*). Các phương pháp truy xuất vector thường ưu tiên độ phủ để đảm bảo không bỏ sót thông tin, dẫn đến việc tập kết quả trả về có thể chứa nhiều tài liệu nhiễu hoặc không sát nghĩa.

Việc tích hợp Re-ranking đóng vai trò như một bộ lọc ngữ nghĩa cao cấp. Nhờ khả năng “đọc hiểu” sâu sắc mối quan hệ giữa câu hỏi và văn bản, Re-ranking giúp loại bỏ các kết quả dương tính giả (*false positives*) và sắp xếp các tài liệu thực sự hữu ích lên vị trí đầu tiên. Các nghiên cứu thực nghiệm đã chứng minh rằng kiến trúc “Truy xuất rồi Xếp hạng lại” (*Retrieve-then-Rerank*) mang lại hiệu suất vượt trội trên các bảng xếp hạng đánh giá truy xuất thông tin, đặc biệt trong các trường hợp truy vấn đòi hỏi khả năng suy luận phức tạp hoặc miền dữ liệu đặc thù [16].

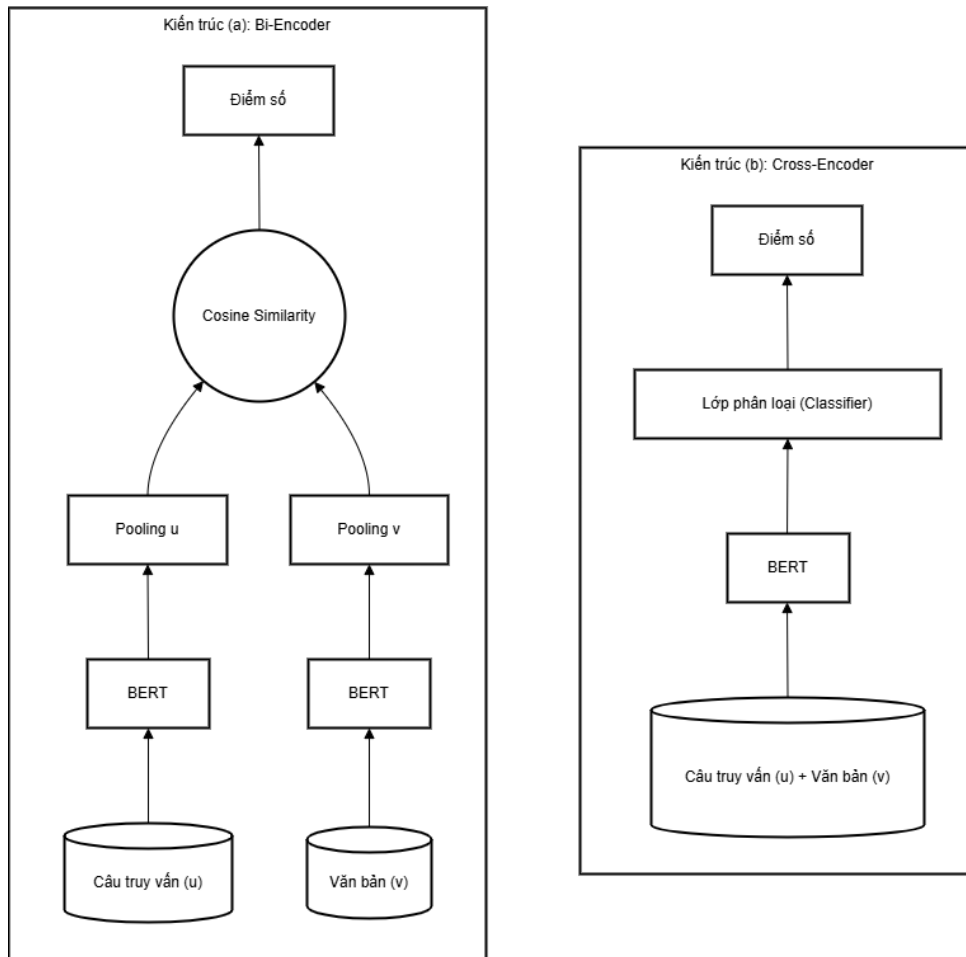


Figure 3.1: So sánh kiến trúc Bi-Encoder và Cross-Encoder

3.7 Cơ sở tri thức và Cơ sở dữ liệu Vector

3.7.1 Cơ sở tri thức (Knowledge Base)

Trong các hệ thống trí tuệ nhân tạo hiện đại, Cơ sở tri thức (*Knowledge Base - KB*) đóng vai trò là kho lưu trữ thông tin tập trung, chứa các dữ liệu, sự kiện và quy tắc mà hệ thống sử dụng để thực hiện việc suy luận hoặc ra quyết định. Đối với các bài toán xử lý ngôn ngữ tự nhiên, cơ sở tri thức thường tồn tại dưới dạng dữ liệu phi cấu trúc (*unstructured data*) như văn bản, tài liệu PDF, hoặc biên bản cuộc họp [13].

Khác với các hệ thống cơ sở dữ liệu quan hệ truyền thống vốn yêu cầu dữ liệu phải được tổ chức chặt chẽ theo hàng và cột, cơ sở tri thức trong các ứng dụng AI chấp nhận sự đa dạng của dữ liệu tự nhiên. Tuy nhiên, thách thức lớn nhất là làm sao để truy xuất thông tin một cách hiệu quả dựa trên ý nghĩa nội dung thay vì chỉ so khớp từ khóa đơn thuần. Đây chính là tiền đề cho sự ra đời của các công nghệ lưu trữ vector.

3.7.2 Cơ sở dữ liệu Vector (Vector Database)

Để giải quyết bài toán quản lý và truy vấn cơ sở tri thức phi cấu trúc, Cơ sở dữ liệu Vector (*Vector Database*) được sử dụng như một giải pháp chuyên dụng. Hệ thống này được thiết kế tối ưu để lưu trữ dữ liệu dưới dạng các vector nhiều chiều (*embeddings*). Chức năng cốt lõi của nó là thực hiện **Tìm kiếm tương đồng (Similarity Search)**, tìm kiếm các vector trong không gian lưu trữ có khoảng cách hình học gần nhất với vector truy vấn thay vì tìm kiếm chính xác tuyệt đối [3].

Các độ đo khoảng cách phổ biến bao gồm Khoảng cách O-clít (*Euclidean Distance*), Tích vô hướng (*Dot Product*) và đặc biệt là Độ tương đồng Cosine (*Cosine Similarity*) thường được dùng trong xử lý văn bản để đo độ tương đồng về ngữ nghĩa bất kể độ dài văn bản.

3.7.3 Mô hình biểu diễn văn bản (Text Embeddings)

Để lưu trữ văn bản vào cơ sở dữ liệu vector, cần có một cơ chế chuyển đổi ngôn ngữ tự nhiên thành các vector số học, gọi là quá trình *Text Embedding*.

Khác với các phương pháp thống kê truyền thống (như TF-IDF hay Bag-of-Words) vốn không nắm bắt được ngữ nghĩa và gặp vấn đề thừa thớt dữ liệu, các mô hình Text Embedding hiện đại thường dựa trên kiến trúc Transformer (như BERT, RoBERTa) [12]. Các mô hình này, đặc biệt là các kiến trúc *Dense Retrieval* (truy xuất dày đặc), có khả năng ánh xạ các câu văn có ý nghĩa tương đồng vào vị trí gần nhau trong không gian vector nhiều chiều, ngay cả khi chúng không chia sẻ từ vựng chung. Đây là thành phần then chốt quyết định độ chính xác của quá trình tìm kiếm ngữ nghĩa trong hệ thống RAG.

3.7.4 Kỹ thuật chỉ mục (Indexing)

Để đảm bảo tốc độ truy vấn trên các tập cơ sở tri thức lớn, việc so sánh vector truy vấn với toàn bộ dữ liệu (phương pháp vét cạn) là không khả thi. Do đó, các kỹ thuật chỉ mục Approximate Nearest Neighbor (ANN) như Hierarchical Navigable Small World (HNSW) (*Hierarchical Navigable Small World*) được áp dụng. HNSW tổ chức dữ liệu thành cấu trúc đồ thị phân cấp, cho phép thuật toán nhanh chóng định vị vùng không gian chứa thông tin liên quan với độ phức tạp tính toán thấp (thường là logarit), đảm bảo khả

năng phản hồi thời gian thực [8].

3.8 Cơ sở lý thuyết về RAG

3.8.1 Tổng quan về Mô hình Ngôn ngữ Lớn (LLM)

Mô hình ngôn ngữ lớn (*Large Language Model - LLM*) đại diện cho bước tiến quan trọng nhất trong lĩnh vực Xử lý ngôn ngữ tự nhiên (NLP) hiện đại. Về bản chất, đây là các mô hình học sâu (*Deep Learning*) được xây dựng dựa trên kiến trúc Transformer, đặc trưng bởi cơ chế sự chú ý (*Self-Attention*) giúp mô hình nắm bắt được các mối quan hệ ngữ nghĩa xa và phức tạp trong văn bản [18].

Các mô hình này (như GPT-4, Gemini, Llama) được huấn luyện trên hàng tỷ tham số với lượng dữ liệu văn bản khổng lồ (thường là quy mô toàn bộ Internet) [15]. Cơ chế hoạt động cốt lõi của LLM là dự đoán từ tiếp theo (*next-token prediction*) dựa trên xác suất thống kê từ chuỗi văn bản đầu vào. Chính nhờ quy mô dữ liệu và kiến trúc này, LLM sở hữu khả năng vượt trội trong việc hiểu ngữ cảnh, tóm tắt văn bản, dịch thuật và lập luận logic, tạo tiền đề cho các ứng dụng AI tạo sinh (*Generative AI*).

3.8.2 Hạn chế của LLM: Vấn đề ảo giác và Tri thức tĩnh

Mặc dù sở hữu năng lực xử lý ngôn ngữ mạnh mẽ như đã đề cập, các mô hình LLM vẫn tồn tại những hạn chế cố hữu đã được cộng đồng nghiên cứu và doanh nghiệp ghi nhận. Vấn đề nghiêm trọng nhất là hiện tượng “ảo giác” (*hallucinations*) - xu hướng của mô hình trong việc tạo ra các câu trả lời với văn phong tự tin, trôi chảy nhưng lại sai lệch hoàn toàn về mặt thực tế hoặc không liên quan đến truy vấn [4].

Nguyên nhân chính của vấn đề này xuất phát từ bản chất “học xác suất” của mô hình: LLM không thực sự “biết” sự thật, mà chỉ dự đoán chuỗi từ ngữ hợp lý nhất. Thêm vào đó, LLM phụ thuộc hoàn toàn vào dữ liệu huấn luyện tĩnh (*Static Knowledge*) [15]. Khi đối mặt với các câu hỏi đòi hỏi thông tin cập nhật mới nhất hoặc dữ liệu đặc thù của doanh nghiệp (như thông số sản phẩm, chính sách nội bộ), mô hình thường thất bại trong việc đưa ra câu trả lời chính xác do thiếu ngữ cảnh.

3.8.3 Giải pháp RAG: Neo giữ thông tin và Kiến trúc lai

Kiến trúc Retrieval-Augmented Generation (RAG) ra đời để giải quyết trực tiếp các hạn chế của LLM bằng cơ chế *neo giữ (grounding)* đầu ra của mô hình vào các nguồn tri thức bên ngoài tin cậy. Cách tiếp cận này mang lại hiệu quả vượt trội về độ chính xác. Các báo cáo từ khối doanh nghiệp chỉ ra rằng việc áp dụng RAG có thể nâng độ chính xác của các truy vấn nghiệp vụ lên trên 90%, biến các mô hình ngôn ngữ tổng quát thành các trợ lý chuyên gia đáng tin cậy.

Định nghĩa hình thức và Cơ chế hoạt động

Về mặt bản chất kỹ thuật, theo định nghĩa gốc của Lewis và cộng sự [6], RAG là một kiến trúc lai (*hybrid architecture*) kết hợp sức mạnh của hai loại bộ nhớ:

1. **Bộ nhớ tham số (Parametric Memory):** Là các trọng số đã được huấn luyện trước của LLM, chứa khả năng ngôn ngữ và tri thức tổng quát.
2. **Bộ nhớ phi tham số (Non-parametric Memory):** Là một chỉ mục vector chứa dữ liệu bên ngoài (biên bản cuộc họp, tài liệu), có thể cập nhật thời gian thực mà không cần huấn luyện lại mô hình [6].

Dưới góc độ xác suất, với một truy vấn đầu vào x , RAG tính toán xác suất của câu trả lời y bằng cách biên duyên hóa (*marginalize*) qua các tài liệu z được tìm thấy [6]:

$$p_{RAG}(y|x) \approx \sum_{z \in TopK(x)} p_{\eta}(z|x) \cdot p_{\theta}(y|x, z) \quad (3.1)$$

Trong đó, hệ thống được cấu thành từ hai phân hệ cốt lõi hoạt động tuần tự:

1. **Bộ truy xuất (The Retriever - p_{η}):** Chịu trách nhiệm ánh xạ câu hỏi x và tài liệu d vào cùng một không gian vector. Độ tương đồng được tính toán (thường bằng tích vô hướng hoặc Cosine) để tìm ra top-k tài liệu liên quan nhất, thường sử dụng kỹ thuật *Dense Passage Retrieval* [5]:

$$score(x, d) = Encoder_Q(x)^T \cdot Encoder_D(d) \quad (3.2)$$

2. Bộ tạo sinh (The Generator - p_θ): Là mô hình ngôn ngữ nhận đầu vào là sự kết hợp giữa truy vấn gốc và ngữ cảnh vừa tìm được ($Input = x \oplus z$). Nhờ cơ chế Attention và kỹ thuật Prompt Engineering [7], mô hình sẽ sử dụng thông tin trong z làm cơ sở dữ liệu để sinh ra câu trả lời y chính xác và có thể kiểm chứng.

3.8.4 Ứng dụng thực tiễn và Bảo mật dữ liệu

Tầm quan trọng của RAG còn được thể hiện qua khả năng khai thác an toàn dữ liệu độc quyền (*proprietary data*).

Nhờ khả năng chuyển đổi LLM từ trạng thái chung chung sang trạng thái “nhận thức ngữ cảnh” (*context-aware*), RAG hiện đã trở thành phương pháp tiếp cận tiêu chuẩn của ngành công nghiệp (*industry-standard approach*).

Table 3.1: So sánh hiệu quả giữa LLM thuần túy và RAG trong doanh nghiệp

Tiêu chí	LLM Thuần túy	RAG (Đồ án áp dụng)
Độ chính xác	Dễ gặp ảo giác [4], thông tin chung chung.	Cao ($> 90\%$), được neo giữ bởi dữ liệu thực.
Cập nhật kiến thức	Bị giới hạn bởi thời điểm huấn luyện (Knowledge Cutoff) [15].	Cập nhật tức thời theo thời gian thực (Real-time).
Chi phí vận hành	Tốn kém nếu phải Retrain liên tục.	Thấp, chỉ cần cập nhật Vector DB.
Dữ liệu riêng tư	Rủi ro lộ lọt nếu dùng để huấn luyện.	An toàn, dữ liệu nằm trong hệ thống kiểm soát nội bộ.

3.9 Truy xuất và Suy luận đa bước (Multi-step Retrieval and Reasoning)

3.9.1 Thách thức của các truy vấn đa bước (Multi-hop queries)

Trong các hệ thống RAG tiêu chuẩn, quá trình truy xuất thường được thực hiện thông qua một thao tác tìm kiếm duy nhất ngay sau khi tiếp nhận truy vấn [6]. Mặc dù cách tiếp cận này thể hiện hiệu năng cao đối với các câu hỏi tra cứu thông tin trực tiếp, các nghiên cứu gần đây chỉ ra rằng nó gặp hạn chế lớn khi xử lý các truy vấn đa bước (*multi-hop queries*) đòi hỏi nhiều tri thức [20]. Các truy vấn đa bước yêu cầu hệ thống phải xác định một thông tin trung gian, sau đó sử dụng thông tin này như một manh mối (*bridge entity*) để tiếp tục định vị đáp án cuối cùng [21]. Việc chỉ truy xuất một lần dựa trên truy vấn gốc thường dẫn đến hiện tượng "khoảng trống ngữ nghĩa" (*semantic gap*), khiến LLM không thu thập đủ ngữ cảnh cần thiết để đưa ra suy luận chính xác [20].

3.9.2 Kỹ thuật Chuỗi suy luận (Chain-of-Thought - CoT)

Để giải quyết bài toán suy luận phức tạp, kỹ thuật Chuỗi suy luận (Chain-of-Thought - CoT) đã được đề xuất nhằm hướng dẫn LLM phân rã một bài toán lớn thành các bước lập luận nhỏ, được thực hiện tuần tự trước khi đưa ra kết quả cuối cùng [19]. Tuy nhiên, Trivedi và cộng sự [17] lập luận rằng CoT truyền thống phụ thuộc hoàn toàn vào bộ nhớ tham số (*parametric memory*) của mô hình. Đối với các bài toán yêu cầu tri thức cập nhật hoặc dữ liệu nội bộ đặc thù (như biên bản hội họp), việc sử dụng CoT thuần túy làm tăng rủi ro phát sinh hiện tượng ảo giác (*hallucination*), do LLM có xu hướng tự sinh ra các bước suy luận thiếu căn cứ thực tế [4].

3.9.3 Phương pháp Lồng ghép Truy xuất và Suy luận (IRCoT)

Nhằm khắc phục những hạn chế của cả RAG tiêu chuẩn và CoT, phương pháp Lồng ghép Truy xuất và Suy luận (Interleaving Retrieval with Chain-of-Thought - IRCoT) đã được giới thiệu [17]. Trái ngược với kiến trúc tách rời giữa truy xuất và tạo sinh, IRCoT thiết lập một vòng lặp tương tác liên tục giữa hai thành phần này.

Theo Trivedi và cộng sự [17], cơ chế hoạt động cốt lõi của IRCoT được thực hiện qua các bước lặp:

1. **Khởi tạo:** Dựa trên câu hỏi gốc, LLM tạo ra một bước suy luận khởi đầu (*partial CoT*).
2. **Truy xuất ngữ cảnh:** Bước suy luận này, chứa các manh mối mới, được sử dụng làm truy vấn (*query*) để tìm kiếm tài liệu liên quan trong không gian vector.
3. **Tích hợp:** Các tài liệu vừa được truy xuất sẽ được bổ sung vào bộ đệm ngữ cảnh (*context window*).
4. **Suy luận tiếp nối:** Dựa trên tập ngữ cảnh đã được làm giàu, LLM tiếp tục sinh ra bước suy luận tiếp theo.
5. **Kết thúc:** Vòng lặp này tiếp diễn cho đến khi LLM thu thập đủ thông tin thực tế hoặc đạt đến giới hạn số bước lặp được cấu hình trước [17].

3.9.4 Khả năng ứng dụng trong hệ thống RAG hội thoại

Đặc thù của dữ liệu hội thoại nhiều người tham gia là tính phân mảnh; thông tin thường không xuất hiện liền mạch mà bị phân tán theo thời gian và đan xen giữa các lượt phát biểu của nhiều diễn giả [22]. Việc ứng dụng kiến trúc IRCoT cho phép hệ thống RAG vượt qua giới hạn của tìm kiếm từ khóa tĩnh. Thông qua vòng lặp của IRCoT [17], hệ thống có khả năng tự động bóc tách các điều kiện truy vấn phức tạp (ví dụ: xác định mốc thời gian diễn ra sự kiện trước, sau đó dùng thời gian đó làm manh mối để tìm kiếm phát biểu của diễn giả cụ thể) và truy vết chính xác chuỗi sự kiện. Cơ chế này đóng vai trò then chốt trong việc đáp ứng yêu cầu tổng hợp thông tin và suy luận logic từ các văn bản hội thoại mang nhiễu.

3.10 Hệ thống tóm tắt đa tầng hỗ trợ truy xuất và tìm kiếm thông tin

Trong các hệ thống xử lý hội thoại dài như transcript cuộc họp, việc truy xuất trực tiếp trên toàn bộ dữ liệu gốc thường gặp nhiều khó khăn do kích thước văn bản lớn, mật độ

thông tin không đồng đều và sự tồn tại của nhiều đoạn hội thoại mang ít giá trị ngữ nghĩa. Các cuộc họp thực tế thường chứa nhiều câu xác nhận ngắn, trao đổi mang tính điều phối hoặc các đoạn lặp lại nội dung đã được đề cập trước đó. Nếu toàn bộ transcript đều được sử dụng trực tiếp cho indexing và retrieval, chi phí xử lý sẽ tăng đáng kể trong khi hiệu quả truy xuất không tương ứng.

Để giải quyết vấn đề này, hệ thống xây dựng cơ chế tóm tắt đa tầng nhằm tổ chức lại thông tin theo nhiều mức độ ngữ nghĩa khác nhau. Thay vì chỉ sinh ra một bản tóm tắt duy nhất cho toàn bộ cuộc họp, hệ thống phân tách dữ liệu thành nhiều tầng biểu diễn, trong đó mỗi tầng tập trung vào một mức thông tin riêng biệt. Cách tiếp cận này giúp giảm đáng kể kích thước dữ liệu cần xử lý nhưng vẫn duy trì khả năng bảo toàn nội dung quan trọng phục vụ retrieval và semantic search.

Kiến trúc của hệ thống gồm bốn tầng chính bao gồm transcript layer, participant layer, topic layer và meeting layer. Các tầng này tạo thành một cấu trúc biểu diễn phân cấp, cho phép hệ thống duy trì cả thông tin chi tiết ở mức hội thoại lẫn khả năng biểu diễn ngữ nghĩa cô đọng ở mức tổng quát hơn.

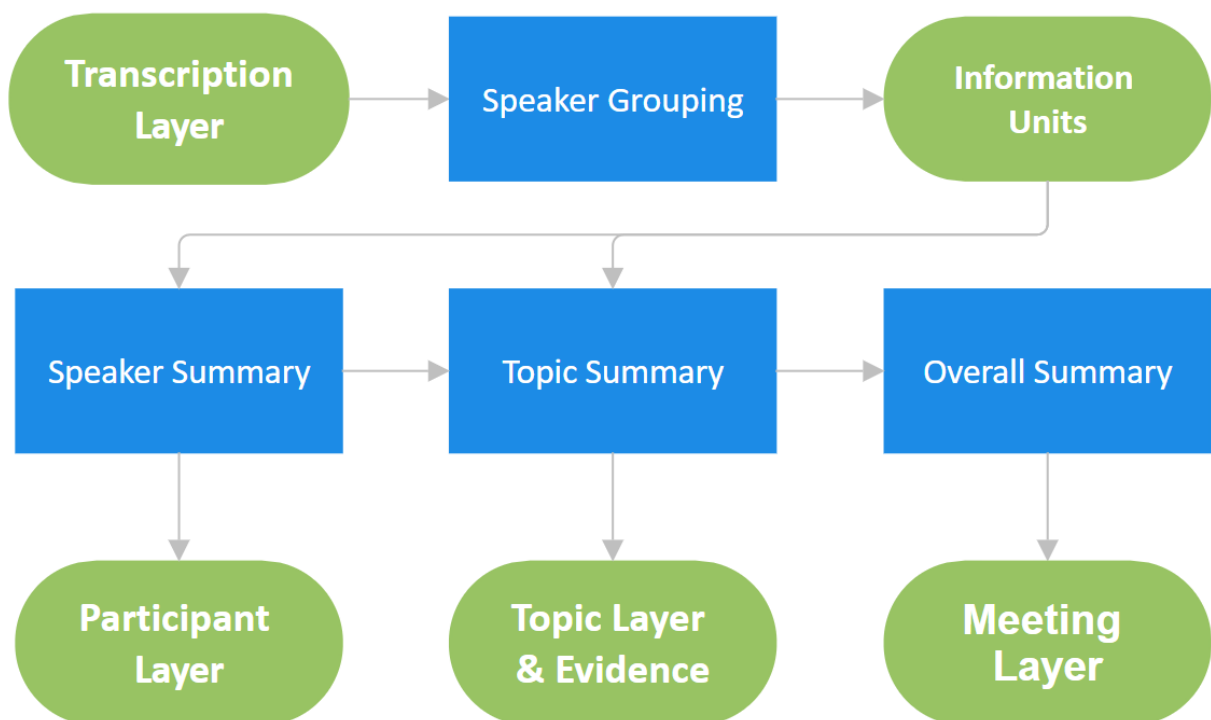


Figure 3.2: Kiến trúc hệ thống tóm tắt đa tầng hỗ trợ truy xuất và tìm kiếm thông tin

3.10.1 Transcript Layer

Transcript layer là tầng dữ liệu gốc của hệ thống và đóng vai trò lưu trữ toàn bộ nội dung hội thoại sau quá trình xử lý audio. Tầng này được xây dựng từ kết quả của pipeline speech recognition và speaker diarization, bao gồm các bước nhận dạng giọng nói, ghép speaker segments, chuẩn hóa văn bản và tái tạo transcript hoàn chỉnh của cuộc họp.

Mỗi đoạn transcript chứa thông tin về speaker, thời gian bắt đầu, thời gian kết thúc và nội dung phát biểu tương ứng. Đây là tầng có mức độ chi tiết cao nhất và phản ánh đầy đủ diễn biến thực tế của cuộc họp.

Tuy nhiên, transcript gốc thường có kích thước rất lớn và chứa nhiều đoạn hội thoại không mang nhiều giá trị truy xuất như các câu xác nhận ngắn, trao đổi mang tính điều phối hoặc các nội dung lặp lại. Việc sử dụng trực tiếp toàn bộ transcript cho retrieval sẽ làm tăng chi phí indexing và similarity search, đặc biệt khi số lượng cuộc họp tăng lên.

Vì vậy, transcript layer chủ yếu được sử dụng như nguồn dữ liệu gốc phục vụ đối chiếu và xác minh thông tin khi cần truy cập ngữ cảnh chi tiết. Đây cũng là tầng cuối cùng giúp hệ thống truy xuất lại đầy đủ nội dung hội thoại thay vì chỉ các biểu diễn cô đọng ở mức tóm tắt.

3.10.2 Participant Layer

Participant layer là tầng tóm tắt được tổ chức theo từng người tham gia trong cuộc họp. Mục tiêu của tầng này là xây dựng biểu diễn ngữ nghĩa tập trung vào đóng góp của từng speaker thay vì toàn bộ diễn biến hội thoại.

Hệ thống thực hiện gom nhóm các phát biểu theo speaker, sau đó trích xuất các information units quan trọng để tạo participant summaries cho từng người tham gia. Quá trình này tập trung giữ lại các nội dung mang giá trị thông tin cao như quyết định, đề xuất, yêu cầu, ràng buộc kỹ thuật hoặc trách nhiệm được giao trong cuộc họp.

Các câu mang tính hội thoại ngắn, xác nhận đơn giản hoặc lặp lại thường được loại bỏ nhằm tăng mật độ thông tin và giảm nhiễu trong dữ liệu tóm tắt. Điều này giúp participant summaries phản ánh rõ hơn vai trò và nội dung đóng góp của từng speaker thay vì lưu lại toàn bộ hội thoại nguyên bản.

Participant layer cho phép hệ thống nhanh chóng xác định người liên quan đến một

chủ đề cụ thể hoặc truy vết các quyết định và đề xuất theo speaker. Ví dụ, hệ thống có thể xác định ai là người đề xuất một giải pháp, ai chịu trách nhiệm cho một tác vụ hoặc ai đã thảo luận về một yêu cầu kỹ thuật cụ thể.

So với transcript layer, participant layer có mức độ cô đọng cao hơn đáng kể nhưng vẫn giữ được khả năng biểu diễn thông tin ở mức chi tiết theo từng người tham gia. Đây là tầng trung gian quan trọng giúp giảm phạm vi tìm kiếm trước khi cần truy cập xuống transcript đầy đủ.

3.10.3 Topic Layer

Topic layer là thành phần trung tâm của hệ thống tóm tắt đa tầng và đóng vai trò biểu diễn nội dung ở mức ngữ nghĩa trung gian của cuộc họp. Khác với participant layer vẫn phụ thuộc vào speaker, topic layer tổ chức thông tin theo các chủ đề thảo luận thay vì theo người phát biểu.

Tầng này được xây dựng bằng cách gom nhóm các information units liên quan, kết hợp các đoạn hội thoại thuộc cùng chủ đề và tạo các topic summaries có mật độ thông tin cao hơn. Mỗi topic summary tập trung mô tả nội dung thảo luận chính, các quyết định quan trọng, ràng buộc kỹ thuật, yêu cầu thiết kế, các đánh đổi giữa nhiều lựa chọn và kết luận liên quan đến chủ đề đó.

Do không còn phụ thuộc vào cấu trúc hội thoại gốc theo speaker, topic summaries có khả năng biểu diễn ngữ nghĩa cô đọng hơn participant summaries trong khi vẫn giữ được phần lớn nội dung quan trọng của cuộc họp.

Topic layer đặc biệt phù hợp cho các tác vụ retrieval và semantic search ở quy mô lớn do khả năng biểu diễn nội dung ở mức chủ đề thay vì mức phát biểu đơn lẻ. Việc sử dụng topic summaries cũng giúp giảm đáng kể số lượng văn bản cần indexing và similarity matching so với transcript gốc.

Ngoài ra, tầng này còn hỗ trợ phát hiện các cuộc họp có nội dung tương tự hoặc các chủ đề xuất hiện lặp lại giữa nhiều tài liệu khác nhau. Trong kiến trúc tổng thể của hệ thống, topic layer đóng vai trò cầu nối giữa transcript chi tiết và meeting summaries ở mức tổng quát.

3.10.4 Meeting Layer

Meeting layer là tầng tóm tắt tổng quát nhất của toàn bộ cuộc họp. Không giống transcript layer được xây dựng trực tiếp từ hội thoại gốc, meeting summaries được sinh ra từ các topic summaries đã được cô đọng ở tầng trước đó.

Tầng này tập trung giữ lại các quyết định chính, mục tiêu của cuộc họp, các yêu cầu quan trọng, vấn đề nổi bật và các kết luận tổng quát. Nhờ được xây dựng trên nền topic summaries thay vì transcript đầy đủ, meeting layer đạt mức độ nén thông tin rất cao nhưng vẫn duy trì khả năng biểu diễn các nội dung cốt lõi của cuộc họp.

Meeting summaries giúp biểu diễn nhanh nội dung tổng quát của một cuộc họp mà không cần truy cập toàn bộ transcript chi tiết. Điều này đặc biệt hữu ích khi xử lý số lượng lớn tài liệu hoặc cần thực hiện indexing ở mức tổng quát.

So với các tầng còn lại, meeting layer có mức độ cô đọng cao nhất và đóng vai trò như lớp biểu diễn ngữ nghĩa tổng quan của toàn bộ tài liệu.

3.10.5 Đặc điểm của hệ thống tóm tắt đa tầng

Hệ thống được thiết kế theo hướng ưu tiên bảo toàn thông tin quan trọng thay vì tối đa hóa mức độ rút gọn văn bản bằng mọi giá. Trong bối cảnh retrieval và semantic search, việc làm mất các information units quan trọng thường gây ảnh hưởng lớn hơn so với việc giữ lại thêm một lượng nhỏ nội dung dư thừa.

Các prompt của hệ thống được xây dựng nhằm hạn chế hiện tượng hallucination và duy trì độ bám sát với transcript gốc. Thay vì paraphrasing mạnh hoặc sinh lại nội dung theo hướng tự do, hệ thống ưu tiên extraction và giữ nguyên các cụm từ quan trọng khi có thể. Cách tiếp cận này giúp tăng độ tin cậy của dữ liệu tóm tắt và giảm nguy cơ sinh ra các thông tin không tồn tại trong transcript ban đầu.

Ngoài chức năng rút gọn dữ liệu, các tầng tóm tắt còn đóng vai trò như các biểu diễn trung gian phục vụ retrieval và semantic indexing. Nhờ việc tổ chức thông tin theo nhiều mức độ ngữ nghĩa khác nhau, hệ thống có khả năng duy trì cả tính cô đọng lẫn khả năng truy xuất chi tiết trên các tập dữ liệu hội thoại lớn.

3.11 Số liệu đánh giá

3.11.1 Các chỉ số đánh giá (Evaluation Metrics)

Phần này trình bày các chỉ số đánh giá được sử dụng trong toàn bộ hệ thống, bao phủ từ mức tín hiệu (speech processing) đến mức suy luận ngữ nghĩa (RAG). Các chỉ số được thiết kế nhằm phản ánh đầy đủ các tiêu chí: độ chính xác, khả năng bảo toàn thông tin, và hiệu quả truy xuất.

Diarization Metrics

Vai trò:

Các chỉ số diarization được sử dụng để đánh giá chất lượng của mô-đun định danh người nói trong pipeline chuyển đổi âm thanh thành văn bản. Đây là bước nền tảng ảnh hưởng trực tiếp đến toàn bộ các thành phần phía sau như speech recognition, summarization, retrieval và question answering. Nếu speaker bị gán sai hoặc nội dung hội thoại bị mất trong giai đoạn diarization, các lỗi này sẽ tiếp tục lan truyền sang các tầng xử lý downstream và làm giảm chất lượng của toàn bộ hệ thống.

Diarization Error Rate (DER):

DER là chỉ số tiêu chuẩn được sử dụng để đo lường mức độ sai lệch giữa kết quả diarization và ground truth theo trục thời gian của hội thoại.

$$DER = \frac{Miss + FalseAlarm + Confusion}{Total\ Speech\ Time}$$

Trong đó:

- **Miss (Missed Speech):** phần speech tồn tại trong audio nhưng không được hệ thống phát hiện.
- **False Alarm:** phần không phải speech nhưng bị nhận diện nhầm là speech.
- **Confusion:** phần speech được phát hiện nhưng bị gán sai speaker.

Trong phạm vi nghiên cứu này, quá trình phân tích tập trung chủ yếu vào hai thành phần Miss và Confusion. Miss làm mất thông tin đầu vào cho ASR và các bước xử lý phía sau, trong khi Confusion ảnh hưởng trực tiếp đến tính đúng đắn của speaker metadata.

Điều này đặc biệt quan trọng đối với các hệ thống retrieval theo người tham gia hoặc các tác vụ tổng hợp thông tin theo speaker.

Speaker Matching (Label Alignment):

Do các speaker labels trong hệ thống diarization chỉ mang tính ký hiệu và không có ý nghĩa cố định, cần thực hiện bước ánh xạ nhãn giữa kết quả dự đoán và ground truth trước khi tính DER.

Bài toán này được mô hình hóa như một bài toán gán tối ưu và được giải bằng thuật toán Hungarian dựa trên confusion matrix ở mức frame. Quá trình alignment giúp đảm bảo việc tính toán DER phản ánh đúng chất lượng speaker assignment thay vì bị ảnh hưởng bởi sự khác biệt trong cách đánh số nhãn speaker.

Phân tích lỗi:

Ngoài các chỉ số định lượng, hệ thống còn thực hiện phân tích các dạng lỗi phổ biến trong speaker diarization nhằm hỗ trợ quá trình chẩn đoán và cải thiện pipeline.

- **Label Oscillation:** hiện tượng speaker labels thay đổi liên tục trong cùng một đoạn phát biểu.
- **Over-merging:** nhiều speaker khác nhau bị gộp thành cùng một nhãn.
- **Over-splitting:** một speaker bị tách thành nhiều nhãn khác nhau.

Các phân tích này được sử dụng như công cụ hỗ trợ đánh giá hành vi của hệ thống và xác định nguyên nhân lỗi trong quá trình diarization, thay vì đóng vai trò như các chỉ số định lượng độc lập.

Speaker-Aware Coverage (SAC):

Speaker-Aware Coverage (SAC) là chỉ số được sử dụng để đánh giá đồng thời khả năng bảo toàn nội dung hội thoại và độ chính xác của speaker attribution trong transcript đầu ra. Khác với DER chủ yếu đánh giá lỗi diarization theo góc nhìn timeline và speaker segmentation, SAC tập trung vào chất lượng của transcript cuối cùng ở mức nội dung và ngữ nghĩa.

SAC được xây dựng từ hai thành phần chính gồm Coverage và Speaker Correctness.

- **Coverage:** tỷ lệ các từ hoặc cụm từ mang ý nghĩa trong Ground Truth vẫn xuất hiện trong transcript đầu ra sau quá trình xử lý.

- **Speaker Correctness:** tỷ lệ nội dung được gán đúng speaker tương ứng.

Chỉ số SAC chỉ đạt giá trị cao khi hệ thống vừa giữ lại được nội dung hội thoại, vừa duy trì đúng mối liên hệ giữa nội dung và speaker metadata. Nếu transcript vẫn giữ được phần lớn nội dung nhưng speaker labels bị gán sai, hoặc ngược lại, SAC sẽ giảm đáng kể.

Điểm khác biệt quan trọng giữa DER và SAC nằm ở mục tiêu đánh giá. DER phản ánh lỗi diarization theo trục thời gian của audio, trong khi SAC phản ánh mức độ mà transcript cuối cùng vẫn bảo toàn đúng nội dung gắn với đúng speaker. Vì vậy, SAC phản ánh trực tiếp hơn tác động của lỗi diarization đối với các tác vụ downstream như retrieval theo participant, meeting summarization hoặc phân tích đóng góp của từng người tham gia trong cuộc họp.

Summarization Metrics

Vai trò:

Các chỉ số summarization được sử dụng để đánh giá khả năng của hệ thống trong việc rút gọn dữ liệu hội thoại dài thành biểu diễn cô đọng hơn nhưng vẫn giữ được các thông tin quan trọng phục vụ cho retrieval và indexing. Trong bối cảnh hệ thống tóm tắt đa tầng, việc đánh giá không chỉ tập trung vào độ ngắn gọn của bản tóm tắt mà còn cần xem xét khả năng bảo toàn nội dung và mức độ bám sát transcript gốc.

Information Coverage Score (ICS):

Information Coverage Score (ICS) được sử dụng để đo lường mức độ bảo toàn thông tin của hệ thống tóm tắt. Chỉ số này đánh giá xem các ý chính, quyết định, yêu cầu hoặc thông tin quan trọng trong ground truth có còn xuất hiện trong bản tóm tắt sinh ra hay không.

Quá trình đánh giá được thực hiện bằng cách phân tách nội dung thành các information units nhỏ, sau đó thực hiện semantic matching giữa ground truth và bản tóm tắt sinh ra thông qua embedding similarity. Hệ thống đồng thời kết hợp lexical overlap nhằm tăng độ ổn định của quá trình đánh giá coverage.

ICS phản ánh khả năng duy trì thông tin ở nhiều tầng summarization khác nhau như participant-level, topic-level và meeting-level. Giá trị ICS càng cao cho thấy bản tóm tắt càng giữ lại được nhiều nội dung quan trọng của cuộc họp.

Compression Ratio (CR):

Compression Ratio (CR) được sử dụng để đánh giá khả năng nén thông tin của hệ thống tóm tắt.

$$CR = \frac{Summary\ Length}{Original\ Transcript\ Length}$$

Chỉ số này phản ánh tỷ lệ giữa độ dài của bản tóm tắt đầu ra và transcript gốc. Giá trị CR thấp cho thấy hệ thống có khả năng rút gọn đáng kể kích thước dữ liệu cần lưu trữ và xử lý, đồng thời vẫn duy trì được lượng thông tin cần thiết phục vụ retrieval.

Trong bối cảnh hệ thống quản lý nhiều cuộc họp, khả năng giảm kích thước dữ liệu nhưng vẫn giữ được coverage là yếu tố quan trọng giúp giảm chi phí indexing và tăng tốc quá trình retrieval. **Hallucination Rate (HR):**

Hallucination Rate (HR) được sử dụng để đánh giá mức độ xuất hiện của các thông tin không tồn tại trong transcript gốc.

Trong quá trình đánh giá, mỗi phát biểu trong bản tóm tắt sẽ được đối chiếu với transcript thông qua semantic similarity và lexical overlap. Nếu một phát biểu không đạt đủ mức tương đồng với nội dung gốc thì sẽ được xem là hallucination.

HR thấp cho thấy nội dung tóm tắt vẫn bám sát transcript ban đầu và hạn chế việc sinh ra các thông tin sai lệch hoặc không có thật. Điều này đặc biệt quan trọng khi bản tóm tắt được sử dụng như lớp trung gian phục vụ retrieval và indexing trong các hệ thống quản lý cuộc họp đa tài liệu.

Các chỉ số đánh giá hệ thống RAG

Để đánh giá toàn diện kiến trúc RAG, hệ thống sử dụng bộ chỉ số đo lường phân tách theo hai giai đoạn cốt lõi: Giai đoạn Truy xuất (Retrieval) và Giai đoạn Tạo sinh (Generation). Việc phân tách này giúp cô lập lỗi và xác định chính xác nguyên nhân làm giảm hiệu năng của hệ thống nằm ở khâu tìm kiếm ngữ cảnh hay khâu suy luận của Mô hình Ngôn ngữ Lớn (LLM).

Đánh giá Mô-đun Truy xuất (Retrieval Metrics) Các chỉ số này tập trung đo lường độ chính xác của cơ sở dữ liệu vector và thuật toán tìm kiếm:

Hit Rate@K (Tỷ lệ trúng đích trong Top K): Ý nghĩa: Đo lường tỷ lệ phần trăm các truy vấn mà hệ thống truy xuất thành công đoạn hội thoại chứa đáp án đúng nằm trong

top K kết quả trả về.

Mục đích: Kiểm chứng hiệu quả của chiến lược phân mảnh văn bản (*Parent Document Retrieval*). So với phương pháp chia *chunk* nguyên câu truyền thống, việc băm nhỏ câu thoại thành các đoạn 40 từ với độ chồng lấp 10 từ được kỳ vọng sẽ làm tăng đáng kể chỉ số này nhờ giảm thiểu tỷ lệ bỏ sót thông tin (High Recall).

Mean Reciprocal Rank - MRR (Điểm xếp hạng nghịch đảo trung bình):

$$MRR = \frac{1}{Q} \sum_{i=1}^Q \frac{1}{rank_i}$$

Ý nghĩa: Đánh giá chất lượng thứ hạng của đoạn văn bản chứa đáp án đúng (Ví dụ: đoạn văn bản đúng nằm ở Top 1 được 1 điểm, Top 2 được 0.5 điểm).

Mục đích: Đảm bảo bộ tái xếp hạng (*Cross-Encoder Re-ranker*) hoạt động hiệu quả, đẩy các ngữ cảnh quan trọng nhất lên đầu danh sách nhằm giảm tải dữ liệu "rác" (nhiều) đưa vào LLM.

Context Relevance (Độ liên quan của Ngữ cảnh): Ý nghĩa: Đo lường tỷ lệ giữa các thông tin thực sự hữu ích so với tổng lượng thông tin có trong cửa sổ ngữ cảnh (*Context Window*).

Mục đích: Khẳng định việc áp dụng kỹ thuật cửa sổ trượt (*Sliding Window*) và mở rộng ngữ cảnh linh hoạt cung cấp vừa đủ lượng thông tin cần thiết cho LLM suy luận mà không làm loãng chủ đề truy vấn.

Đánh giá Mô-đun Tạo sinh (Generation Metrics) Do đặc thù của dữ liệu âm thanh mang nhiều sai số (ASR Noise), việc đánh giá chất lượng câu trả lời cuối cùng do LLM sinh ra là bắt buộc nhằm kiểm chứng khả năng tự sửa lỗi và chống ảo giác:

Faithfulness (Tính xác thực / Độ bám sát ngữ cảnh): Ý nghĩa: Đo lường xem câu trả lời của LLM có hoàn toàn dựa trên ngữ cảnh đã được truy xuất hay không, hay mô hình tự bịa đặt thêm thông tin.

Mục đích: Đảm bảo tính trung thực của hệ thống. Một câu trả lời có thể đúng với kiến thức thực tế bên ngoài nhưng nếu không xuất phát từ biên bản cuộc họp thì vẫn bị đánh giá điểm Faithfulness thấp (vi phạm nguyên tắc *grounded*).

Answer Relevance (Độ liên quan của câu trả lời): Ý nghĩa: Đánh giá mức độ trực tiếp và trọn vẹn của câu trả lời so với câu hỏi gốc ban đầu.

Mục đích: Đo lường khả năng lập luận đa bước (*multi-step reasoning*) của hệ thống. Đối với các câu hỏi phức tạp, hệ thống phải tổng hợp đủ thông tin để đưa ra câu trả lời toàn diện thay vì trả lời vòng vo hoặc thiếu ý.

Đánh giá Hiệu năng Siêu dữ liệu (Metadata Impact) Bên cạnh các chỉ số tiêu chuẩn, đề án bổ sung thêm phương pháp đánh giá thực nghiệm (A/B Testing) nhằm đo lường vai trò của siêu dữ liệu:

Metadata Filtering Efficiency (Hiệu quả lọc siêu dữ liệu): Ý nghĩa: So sánh chéo các chỉ số Hit Rate@K và MRR giữa hai cấu hình hệ thống: truy xuất vector thuần túy so với truy xuất vector kết hợp lọc siêu dữ liệu (định danh người nói, mốc thời gian).

Mục đích: Chứng minh việc tổ chức siêu dữ liệu dạng *payload* giúp loại bỏ triệt để các kết quả trùng lặp từ vụng nhưng khác nguồn phát biểu, đặc biệt quan trọng trong dữ liệu hội thoại đa diễn giả.

Chapter 4

Thiết Kế & Hiện Thực Hệ Thống

4.1 Tổng quan kiến trúc hệ thống

4.1.1 Mô hình kiến trúc tổng thể

Hệ thống được thiết kế theo mô hình đường ống (*Pipeline Architecture*), chia thành các mô-đun xử lý tuần tự nhằm giải quyết bài toán truy xuất thông tin từ dữ liệu âm thanh hội thoại phức tạp. Mục tiêu thiết kế là đảm bảo khả năng xử lý các cuộc họp kéo dài, đa người nói và cung cấp câu trả lời có độ chính xác cao dựa trên ngữ cảnh thực tế.

Kiến trúc tổng thể của hệ thống bao gồm hai mô-đun chính hoạt động liên kết chặt chẽ:

1. **Mô-đun Xử lý tiếng nói (Speech Processing Module):** Chịu trách nhiệm chuyển đổi dữ liệu âm thanh thô thành văn bản có cấu trúc, bao gồm việc định danh người nói (*Speaker Diarization*) và nhận dạng nội dung (*ASR*).
2. **Mô-đun Truy xuất và Tạo sinh tri thức (RAG Module):** Thực hiện việc lưu trữ, đánh chỉ mục vector và sử dụng mô hình ngôn ngữ lớn (*LLM*) để trả lời câu hỏi dựa trên văn bản đã được trích xuất.

Sự tách biệt giữa hai mô-đun này giúp hệ thống đạt được tính độc lập cao, dễ dàng nâng cấp từng thành phần (ví dụ: thay đổi model ASR hoặc model Embedding) mà không ảnh hưởng đến toàn bộ cấu trúc.

4.1.2 Luồng dữ liệu trong hệ thống

Dữ liệu trong hệ thống được luân chuyển và biến đổi qua 4 giai đoạn chính, đảm bảo tính vẹn toàn từ tín hiệu âm thanh đến câu trả lời cuối cùng:

- **Giai đoạn 1: Thu thập và Tiền xử lý (Ingestion).** Đầu vào là các bản ghi âm cuộc họp (Meeting Recordings). Các bản ghi này thường chứa tạp âm và có nhiều người tham gia. Hệ thống thực hiện chuẩn hóa âm thanh để tối ưu cho bước nhận dạng.
- **Giai đoạn 2: Chuyển đổi cấu trúc (Transcribing & Diarization).** Phân hệ ASR xử lý tín hiệu âm thanh để tách biệt các đoạn thoại của từng diễn giả và chuyển đổi thành văn bản. Kết quả đầu ra không chỉ là chuỗi ký tự đơn thuần mà là dữ liệu bán cấu trúc (*Semi-structured Data*) chứa thông tin siêu dữ liệu (*Metadata*) quan trọng: định danh người nói (*Speaker ID*) và mốc thời gian (*Timestamp*).
- **Giai đoạn 3: Đánh chỉ mục tri thức (Indexing).** Dữ liệu văn bản kèm metadata được phân hệ RAG tiếp nhận. Tại đây, kỹ thuật *Chunking* thông minh được áp dụng để chia nhỏ hội thoại nhưng vẫn giữ nguyên ngữ cảnh. Các đoạn văn bản này sau đó được mã hóa thành vector (*Embeddings*) và lưu trữ trong Cơ sở dữ liệu Vector để phục vụ truy vấn.
- **Giai đoạn 4: Truy xuất và Tổng hợp (Retrieval & Generation).** Khi người dùng đặt câu hỏi (ví dụ: “Ai đề xuất phương án A?”), hệ thống thực hiện tìm kiếm ngữ nghĩa (*Semantic Search*) kết hợp xếp hạng lại (*Re-ranking*). Các đoạn hội thoại liên quan nhất được trích xuất và đưa vào LLM để sinh câu trả lời, kèm theo dẫn chứng cụ thể về thời gian và người nói.

4.1.3 Các nguyên lý thiết kế cốt lõi

Dựa trên đặc thù của bài toán xử lý biên bản cuộc họp, hệ thống tuân thủ các nguyên tắc thiết kế sau:

1. **Tính truy vết (Traceability):** Mọi thông tin do LLM sinh ra đều phải được neo giữ (*grounded*) vào dữ liệu gốc. Hệ thống được thiết kế để luôn trả về nguồn gốc

của thông tin (trích đoạn âm thanh tại giây thứ bao nhiêu, do ai nói) để người dùng có thể đối chiếu, đặc biệt quan trọng trong các lĩnh vực pháp lý và y tế.

2. **Khả năng chống nhiễu (Robustness):** Nhận thức được việc transcript từ ASR có thể chứa sai sót nhận dạng, cơ chế tìm kiếm Vector được tối ưu hóa để nắm bắt ý nghĩa ngữ nghĩa tổng quát thay vì chỉ so khớp từ khóa chính xác, giúp hệ thống vẫn tìm được thông tin đúng ngay cả khi văn bản có lỗi sai chính tả nhỏ.
3. **Xử lý ngữ cảnh dài (Long-context Handling):** Với các cuộc họp kéo dài hàng giờ, hệ thống sử dụng chiến lược cửa sổ trượt (*sliding window*) và bộ nhớ đệm hội thoại để đảm bảo LLM không bị mất ngữ cảnh khi xử lý các câu hỏi phức tạp hoặc các yêu cầu tóm tắt đa tầng (*multi-level summarization*).

4.2 Mô hình Xử lý tiếng nói (Speech Processing Module)

Mô hình Xử lý tiếng nói đóng vai trò là lớp chuyển đổi trung gian giữa tín hiệu âm thanh thô và biểu diễn văn bản có cấu trúc, làm cơ sở cho toàn bộ các bước truy xuất tri thức phía sau. Mô hình này được thiết kế để bảo toàn thông tin ngữ cảnh hội thoại, đặc biệt là định danh người nói và mốc thời gian.

Vì vậy, kết quả của mô hình không đơn thuần là transcript, mà là một tập các phân đoạn hội thoại được thêm dữ liệu ngoài hội thoại, đóng vai trò là dữ liệu cơ sở cho quá trình đánh chỉ mục và truy xuất sau này.

Mô hình được phân thành 2 phần chính:

- Định danh người nói (Speaker Diarization).
- Nhận dạng tiếng nói tự động (Automatic Speech Recognition - ASR).

Hai phần này hoạt động tuần tự, kết quả của Diarization quyết định các phân đoạn được đưa vào ASR, đồng thời cung cấp thông tin người nói để gán nhãn cho văn bản được sinh ra. Việc hoạt động tuần tự là cần thiết, vì các mô hình ASR tốn kém chi phí vận hành, xử lý dữ liệu qua Diarization giúp tách biệt lời nói rõ ràng giữa những diễn giả, từ đó hạn chế trộn lẫn ngữ cảnh giữa các đoạn hội thoại và cũng giúp khử nhiễu, giữ cho mô hình ASR hoạt động ổn định hơn so với bản ghi liên tục.

4.2.1 Định danh người nói

Ở bước đầu tiên, tín hiệu âm thanh đầu vào được xử lý bằng mô-đun phát hiện hoạt động giọng nói (VAD) nhằm xác định các khoảng thời gian có lời nói. Các đoạn lời nói sau đó được phân loại thành hai nhóm dựa trên độ dài: các đoạn đủ dài để trích xuất embedding ổn định và các đoạn ngắn không đạt ngưỡng tối thiểu.

Việc tách riêng các đoạn ngắn ngay từ đầu nhằm tránh việc đưa các embedding kém ổn định vào quá trình phân cụm chính, vốn có thể dẫn tới các lỗi gộp hoặc tách nhãn không mong muốn.

Phân cụm thô với Embedding cỡ lớn: Các đoạn giọng nói đủ dài được sử dụng để trích xuất speaker embedding bằng các mô hình chuyên dụng (như ECAPA-TDNN hoặc TitaNet-large). Các mô hình này có tính phân biệt cao và phù hợp cho việc xây dựng cấu trúc phân cụm ổn định trong không gian đặc trưng.

Tập embedding thu được được đưa vào mô-đun phân cụm nhằm ước lượng số lượng người nói và gán nhãn sơ bộ cho các đoạn giọng nói dài. Kết quả của pha này được xem là các cụm thô (coarse clusters), đóng vai trò nền tảng cho các bước xử lý tiếp theo.

Khởi tạo đại diện cụm (Cluster pivots): Sau khi hoàn tất pha phân cụm thô, mỗi cụm người nói được đại diện bởi một tập con các đoạn giọng nói tiêu biểu. Các đoạn này được sử dụng để xây dựng các vector đại diện (pivot embeddings) cho từng cụm, nhằm tạo cầu nối giữa không gian embedding cỡ lớn và không gian embedding ở pha tiếp theo.

Các pivot này cho phép chuyển thông tin về cấu trúc người nói đã được ước lượng sang không gian embedding khác, đồng thời giảm thiểu ảnh hưởng của nhiễu do các đoạn giọng nói ngắn gây ra.

Gán nhãn bổ sung: Các đoạn giọng nói ngắn bị loại khỏi pha phân cụm ban đầu được xử lý riêng. Các đoạn này được trích xuất embedding bằng một mô hình biểu diễn có khả năng xử lý hiệu quả các phân đoạn ngắn và giàu ngữ cảnh cục bộ.

Embedding của các đoạn ngắn được so sánh với các vector đại diện của từng cụm thu được từ gom cụm thô. Nhãn người nói được gán dựa trên độ tương đồng cao nhất, đảm bảo mọi đoạn giọng nói đều được gán nhãn mà không làm thay đổi cấu trúc phân cụm đã được xác định trước đó.

Cuối cùng, các đoạn giọng nói được hợp nhất theo trật tự thời gian để tạo ra kết quả diarization hoàn chỉnh. Pipeline đảm bảo rằng tất cả các đoạn lời nói, bao gồm cả các

đoạn ngắn, đều được gán nhãn người nói tương ứng, đồng thời giữ nguyên cấu trúc phân cụm ổn định từ pha đầu tiên.

Thiết kế hai bước gán nhãn tách biệt rõ ràng nhằm hạn chế ảnh hưởng lỗi từ các đoạn ngắn của những embeddings cỡ lớn đến cấu trúc phân cụm, và tối ưu ưu điểm của hai loại mô hình embeddings hiện đại.

4.2.2 Mô hình Tự động Nhận diện Giọng nói

ASR trong hệ thống không chỉ đảm nhiệm chức năng chuyển đổi tín hiệu âm thanh sang văn bản, mà còn đóng vai trò chuẩn hóa cấu trúc hội thoại để phục vụ các bước xử lý ngữ nghĩa và truy xuất tri thức phía sau.

Gán nhãn người nói cho văn bản ASR Sau khi hoàn tất quá trình Diarization, mỗi phân đoạn giọng nói được gán nhãn và mốc thời gian. Mỗi phân đoạn không được xem là độc lập hoàn toàn trong quá trình nhận dạng tiếng nói. Trong thực tế, các phân đoạn này thường bị phân mảnh do hiện tượng cắt lời, dao động nhàn (oscillation) hoặc tách đoạn quá mức, dẫn đến độ dài ngữ cảnh không tối ưu cho ASR.

Xử lý phân mảnh trước ASR Kết quả phân tích Diarization cho thấy một số lỗi còn tồn đọng như oscillation, merge và split, đặc biệt ở các đoạn hội thoại ngắn liên tiếp. Điều này làm suy giảm hiệu quả trong suy đoán ngữ cảnh của ASR nếu áp dụng trực tiếp lên từng đoạn riêng biệt. Vì vậy, mô hình thực hiện ASR theo từng cụm phân đoạn (segment grouping), trong đó các đoạn gần nhau về mặt thời gian được gom lại thành các đơn vị lớn hơn.

Việc gom nhóm phân đoạn được điều khiển bởi một ngưỡng độ dài tối thiểu, nhằm đảm bảo mỗi đơn vị đầu vào cho ASR chứa đủ thông tin ngữ âm và ngữ cảnh để mô hình hoạt động ổn định. Cách tiếp cận này giúp giảm thiểu các lỗi nhận dạng do ngữ cảnh quá ngắn, đồng thời hạn chế việc phá vỡ cấu trúc hội thoại gốc.

Để cân bằng giữa việc bảo toàn ranh giới hội thoại và yêu cầu về độ dài ngữ cảnh, hệ thống áp dụng chiến lược soft-padding sliding window. Cụ thể, mỗi cụm phân đoạn được mở rộng thêm một khoảng ngữ cảnh ngắn ở hai phía (trước và sau), với các đoạn đệm này không được xem là trung tâm nhận dạng nhưng cung cấp thêm thông tin ngữ cảnh cho mô hình ASR. Sau khi hoàn tất nhận dạng, chỉ phần văn bản tương ứng với vùng trung tâm của cửa sổ được giữ lại, trong khi phần đệm được loại bỏ nhằm tránh

trùng lặp nội dung.

Hiệu chỉnh cấu trúc hội thoại dựa trên mô hình ngôn ngữ lớn Mặc dù chiến lược gom cụm phân đoạn và soft-padding sliding window giúp ổn định đầu vào cho ASR, đầu ra văn bản thu được vẫn có thể chưa phản ánh chính xác cấu trúc hội thoại thực tế. Nguyên nhân là các ranh giới phân đoạn được xác định ở mức tín hiệu âm thanh không luôn tương thích với ranh giới cú pháp và ngữ nghĩa ở mức văn bản, đặc biệt trong các tình huống hội thoại nhanh, cắt lời hoặc phản hồi ngắn.

Do đó, hệ thống áp dụng một bước hậu xử lý dựa trên mô hình ngôn ngữ lớn (LLM) nhằm hiệu chỉnh ranh giới giữa các đoạn hội thoại dựa trên cấu trúc câu và ngữ nghĩa tổng thể của transcript. Mô hình ngôn ngữ trong bước này không sinh thêm nội dung mới, mà chỉ thực hiện việc gộp hoặc tách các đoạn văn bản để đảm bảo tính nhất quán về mặt hội thoại trước khi chuyển sang giai đoạn đánh chỉ mục và truy xuất tri thức.

Bước hiệu chỉnh này được đặt sau phân hệ ASR và được xem là cần thiết để giảm thiểu tác động của hiện tượng phân mảnh phân đoạn và các lỗi tồn đọng từ quá trình diarization, mặc dù hiện chưa có thước đo định lượng trực tiếp cho bước xử lý này.

4.3 Mô hình Truy xuất và Tạo sinh tri thức (RAG Module)

Tiếp nối kết quả từ quá trình chuyển đổi giọng nói sang văn bản, mô-đun RAG đóng vai trò trung tâm trong việc tổ chức, khai thác và tổng hợp tri thức từ dữ liệu hội thoại. Dựa trên đầu vào là các tệp JSON có cấu trúc (chứa thông tin người nói, thời gian và nội dung) từ mô-đun ASR, mô-đun này được thiết kế để thực hiện hai nhóm chức năng cốt lõi: *Truy xuất thông tin theo ngữ cảnh* (phục vụ hỏi đáp) và *Tổng hợp nội dung đa tầng* (phục vụ tóm tắt).

Để hiện thực hóa các chức năng trên, mô-đun vận hành theo một quy trình xử lý khép kín gồm bốn giai đoạn kỹ thuật chính:

1. **Indexing (Đánh chỉ mục):** Mã hóa văn bản thành vector và lưu trữ vào cơ sở dữ liệu chuyên dụng, kèm theo siêu dữ liệu để phục vụ lọc thông tin.
2. **Retrieval (Truy xuất):** Tìm kiếm các đoạn hội thoại liên quan nhất đến câu hỏi dựa

trên ngữ nghĩa (Semantic Search) và xếp hạng lại (Re-ranking) để loại bỏ nhiễu.

3. **Generation (Tạo sinh):** Tích hợp LLM để sinh câu trả lời tự nhiên hoặc bản tóm tắt dựa trên ngữ cảnh đã trích xuất.
4. **Summarization (Tổng hợp):** Áp dụng các kỹ thuật xử lý ngữ cảnh dài để tóm tắt toàn bộ cuộc họp hoặc tóm tắt theo từng đối tượng người nói cụ thể.

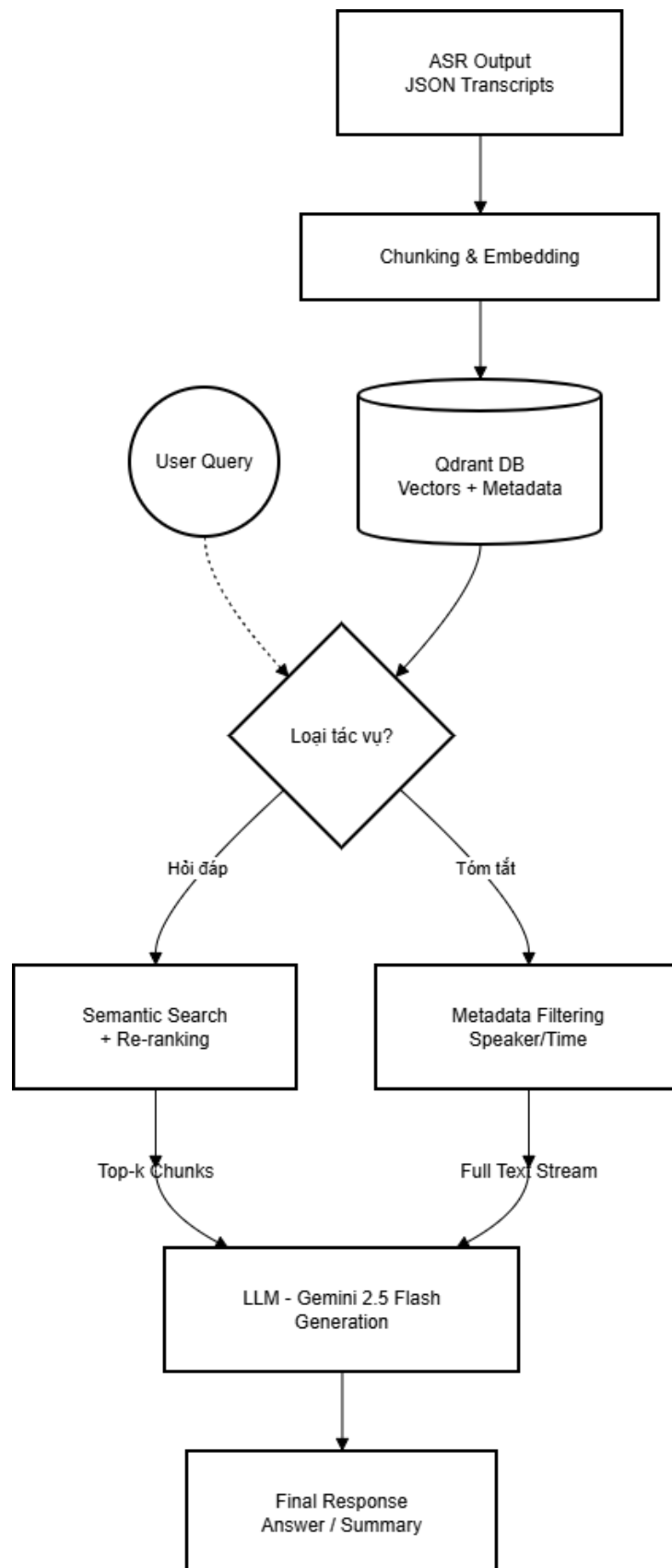


Figure 4.1: Sơ đồ luồng xử lý dữ liệu tích hợp chức năng Tóm tắt trong mô-đun RAG

4.3.1 Xây dựng và Đánh chỉ mục dữ liệu (Indexing)

Giai đoạn đầu tiên của đường ống RAG là chuyển đổi dữ liệu hội thoại thô thành cấu trúc có thể tìm kiếm được. Quá trình này không chỉ đơn thuần là lưu trữ văn bản mà còn bao gồm việc làm sạch dữ liệu, mã hóa ngữ nghĩa (Vectorization) và thiết lập cơ sở dữ liệu vector tối ưu.

Tiền xử lý và Chuẩn hóa dữ liệu đầu vào

Hệ thống tiếp nhận đầu vào là các tệp JSON được sinh ra từ mô-đun ASR. Do đặc thù của dữ liệu hội thoại tự nhiên, thông tin đầu vào thường gặp các vấn đề như thiếu nhân vật nói hoặc định dạng không đồng nhất. Để đảm bảo tính toàn vẹn dữ liệu, quy trình chuẩn hóa được áp dụng trong phương thức `_load_memory_kb` với các tiêu chí sau:

- **Phân đoạn văn bản (Chunking):** Mỗi câu thoại dài được chia nhỏ thành các đoạn (*chunks*) có độ dài tối đa 40 từ với độ chồng lấp (*overlap*) là 10 từ. Việc này giúp vector đại diện tập trung hơn vào các ý chính và tránh hiện tượng mất mát thông tin tại ranh giới các đoạn.
- **Định danh lượt lời:** Mỗi lượt lời (utterance) được gán một định danh duy nhất (`utt_id`) phục vụ truy vết.
- **Xử lý dữ liệu khuyết thiếu:** Hệ thống kiểm tra các trường bắt buộc (`speaker`, `text`). Cơ chế an toàn (*Fail-safe*) sẽ gán giá trị mặc định (ví dụ: "Unknown Speaker") nếu dữ liệu bị thiếu nhằm tránh lỗi thực thi.
- **Tổng hợp toàn văn:** Song song với xử lý từng câu, hệ thống ghép nối nội dung thành chuỗi văn bản liên mạch (`full_text`) phục vụ riêng cho tác vụ tóm tắt.

```
1 def _chunk_text(self, text, chunk_size=40, overlap=10):
2     """Chia văn bản dài thành các đoạn ngắn dựa trên số từ."""
3     words = text.split()
4     # Nếu câu ngắn hơn chunk_size, giữ nguyên
5     if len(words) <= chunk_size:
6         return [text]
7
```

```

8     chunks = []
9     for i in range(0, len(words), chunk_size - overlap):
10         chunk_str = " ".join(words[i:i + chunk_size])
11         chunks.append(chunk_str)
12     return chunks
13
14 def _load_memory_kb(self):
15     # ... (Code for file reading omitted)
16     for i, utt in enumerate(data):
17         utt_id = f"utt_{i}"
18
19         # Gan gia tri mac dinh neu thieu (Fail-safe mechanism)
20         speaker = utt.get('speaker', 'Unknown Speaker')
21         text = utt.get('text', '')
22
23         # Luu tru metadata
24         kb["utterances"][utt_id] = {
25             'speaker': speaker,
26             'text': text,
27             'start': utt.get('start', 0)
28         }
29         full_text_list.append(f"{speaker}: {text}")

```

Listing 4.1: Logic tiền xử lý dữ liệu hội thoại

Cấu hình Cơ sở dữ liệu Vector (Qdrant)

Sau khi dữ liệu được làm sạch, hệ thống sử dụng *Qdrant* để lưu trữ và đánh chỉ mục. Quá trình khởi tạo bao gồm thiết lập thuật toán HNSW và lựa chọn mô hình Embedding phù hợp.

Cấu hình Index (HNSW Configuration): Hệ thống sử dụng thuật toán HNSW để tối ưu hóa tốc độ tìm kiếm. Các tham số cấu hình trong lớp `HnswConfigDiff` được thiết lập dựa trên khuyến nghị thực nghiệm cho dữ liệu văn bản:

- **Độ đo khoảng cách:** Sử dụng `Distance.COSINE` để đo độ tương đồng ngữ nghĩa.
- **Tham số $m = 16$:** Số lượng liên kết tối đa cho mỗi node, giúp cân bằng giữa tốc

độ truy vấn và bộ nhớ.

- **Tham số $ef_construct = 100$:** Độ sâu tìm kiếm khi xây dựng chỉ mục, đảm bảo độ chính xác cao (Recall) ngay từ đầu.

Mã hóa và Lưu trữ (Vectorization & Upsert): Dựa trên lý thuyết về Text Embeddings, hệ thống sử dụng mô hình BAAI/bge-m3 để chuyển đổi văn bản thành vector. Điểm đặc biệt trong kiến trúc lưu trữ là cơ chế *Payload*:

- **Vector:** Chỉ chứa thông tin ngữ nghĩa (`utt['text']`).
- **Payload (Siêu dữ liệu):** Lưu trữ bản gốc (`text_original`), ID và người nói. Điều này cho phép hệ thống truy xuất ngược lại nội dung gốc ngay lập tức mà không cần tra cứu bảng dữ liệu thô bên ngoài.

```
1  # 1. Tao Collection voi cau hinh HNSW
2  self.client.create_collection(
3      collection_name=self.collection_name,
4      vectors_config=VectorParams(size=EMBEDDING_DIM, distance=
5      Distance.COSINE),
6      hnsw_config=HnswConfigDiff(m=16, ef_construct=100)
7  )
8
9  # 2. Vector hoa va chuan bi Payload
10 embeddings = embed_model.encode(docs, show_progress_bar=True)
11
12 points = [
13     PointStruct(
14         id=ids[i],
15         vector=embeddings[i].tolist(),
16         payload={
17             "text_original": utt['text'],
18             "doc_id": uid,
19             "speaker": utt['speaker']
20         }
21     )
22     for i in range(len(ids))
23 ]
```

```

23
24 # 3. Batch Upsert (Nạp dữ liệu theo lô)
25 self.client.upsert(self.collection_name, points[i:i+batch_size])

```

Listing 4.2: Quy trình khởi tạo Collection và nạp dữ liệu

4.3.2 Quy trình Truy xuất đa tầng (Multi-stage Retrieval)

Chiến lược thiết kế

Quy trình truy xuất được xây dựng theo kiến trúc đường ống hai giai đoạn (*Retrieve-then-Rerank*). Chiến lược này nhằm giải quyết bài toán cân bằng giữa hiệu năng và độ chính xác:

- **Giai đoạn Truy xuất (Retrieval):** Sử dụng kiến trúc Bi-Encoder để tìm kiếm nhanh trong không gian vector, chấp nhận độ chính xác tương đối để đảm bảo không bỏ sót thông tin (*High Recall*).
- **Giai đoạn Tái xếp hạng (Re-ranking):** Sử dụng kiến trúc Cross-Encoder để đánh giá sâu ngữ nghĩa từng cặp câu, loại bỏ nhiễu để đạt độ chính xác cao nhất (*High Precision*).

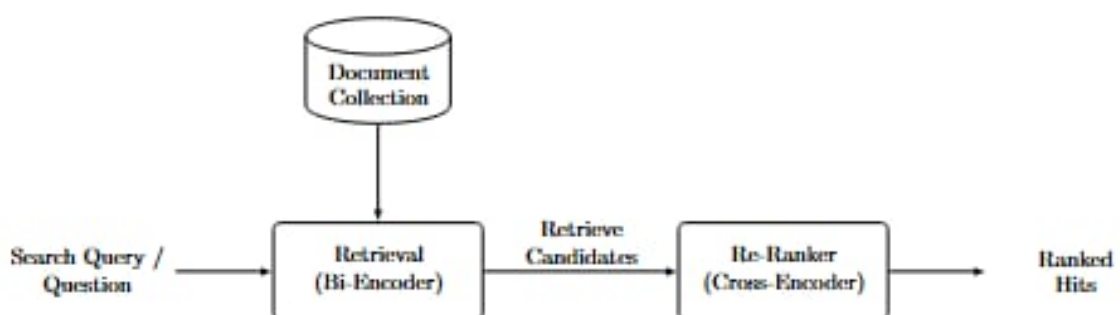


Figure 4.2: Sơ đồ quy trình truy xuất hai giai đoạn (Retrieval & Re-ranking)

Chi tiết hiện thực kỹ thuật

Hệ thống được hiện thực hóa bằng thư viện `sentence-transformers` và `qdrant-client`, trải qua 4 bước xử lý tuần tự:

Bước 1: Tìm kiếm Vector (Candidate Generation): Mục tiêu là thu hẹp không gian tìm kiếm từ toàn bộ cơ sở dữ liệu xuống một tập ứng viên nhỏ.

- **Mã hóa:** Câu truy vấn được vector hóa và chuẩn hóa (`normalize_embeddings=True`) để tương thích với độ đo Cosine.
- **Truy vấn Qdrant:** Hệ thống thực hiện `query_points` với tham số giới hạn `limit=30`. Ngưỡng 30 này đóng vai trò là "Bộ lọc thô", đủ rộng để bao quát các ngữ cảnh tiềm năng.
- **Payload:** Dữ liệu trả về bao gồm cả văn bản gốc (`text_original`) để phục vụ bước tiếp theo.

```
1         query_vec = embed_model.encode([query],
2         normalize_embeddings=True)[0].tolist()
3
4         resp = self.client.query_points(
5             collection_name=self.collection_name,
6             query=query_vec,
7             # query_filter=query_filter,
8             limit=30,
9             with_payload=True
10        )
11
12        hits = resp.points or []
13
14        if not hits: return []
```

Bước 2: Đánh giá ngữ nghĩa chéo (Cross-Encoder Scoring): Đây là bước tinh lọc quan trọng nhất. Hệ thống sử dụng mô hình BAAI/bge-reranker-large — một mô hình SOTA (State-of-the-art) về khả năng rerank đa ngôn ngữ.

- Hệ thống ghép cặp (Câu hỏi, Đoạn văn bản) từ 30 ứng viên ở Bước 1.
- Danh sách các cặp [Query, Document] này sau đó được đưa vào mô hình để thực hiện dự đoán song song (prediction), tạo ra điểm số tương đồng ngữ nghĩa chính xác cho từng ứng viên.

```

1      # 2. Rerank
2          pairs = []
3          for hit in hits:
4              if hit.payload:
5                  pairs.append([query, hit.payload.get('
text_original', '')])
6
7          if not pairs: return []
8
9          scores = rerank_model.predict(pairs) # type: ignore
10

```

Bước 3: Chọn lọc kết quả (Final Selection): Dựa trên điểm số mới từ Cross-Encoder, danh sách được sắp xếp lại (sorted) theo thứ tự giảm dần.

- Hệ thống thực hiện cắt ngưỡng lấy **Top-5** kết quả tốt nhất (`[:5]`).
- **Ý nghĩa:** Con số 5 được lựa chọn thực nghiệm nhằm tối ưu hóa bộ nhớ đệm (Context Window) của LLM, đảm bảo mô hình chỉ nhận được những thông tin tinh túy nhất, giảm thiểu hiện tượng ảo giác do nhiều thông tin.

```

1      # Top 5
2          ranked_hits = sorted(zip(scores, hits), key=lambda x: x
[0], reverse=True)[:5]
3

```

Bước 4: Mở rộng ngữ cảnh (Context Augmentation): Một thách thức đặc thù của dữ liệu hội thoại là tính liên kết ngữ nghĩa giữa các lượt lời. Một câu nói đơn lẻ (ví dụ: *"Tôi đồng ý với ý kiến đó"*) sẽ trở nên vô nghĩa nếu thiếu câu nói trước đó. Để giải quyết vấn đề này, hệ thống áp dụng kỹ thuật **Cửa sổ trượt (Sliding Window)** để mở rộng ngữ cảnh xung quanh kết quả tìm kiếm.

Dựa trên chỉ số (*index*) của đoạn hội thoại tìm được trong chuỗi thời gian gốc, hệ thống thực hiện truy xuất thêm các đoạn lân cận:

- **Cơ chế:** Với mỗi kết quả h_i tại vị trí k , hệ thống sẽ lấy thêm đoạn $k - 1$ (lượt lời trước đó) và $k + 1$ (lượt lời sau đó).
- **Mục đích:** Việc lấy thêm đoạn $k - 1$ giúp giải quyết các tham chiếu đại từ (như "đó", "anh ấy"), trong khi đoạn $k + 1$ cung cấp phản hồi ngay lập tức.
- **Hợp nhất & Khử trùng:** Hệ thống sử dụng một tập hợp `processed_ids` để đảm bảo các đoạn hội thoại không bị lặp lại nếu các kết quả tìm kiếm nằm quá gần nhau.

```

1  # 3. Context Augmentation
2      final_chunks = []
3      processed_ids = set()
4
5      # Map ID to index
6      id_to_index = {uid: i for i, uid in enumerate(self.
kb_memory["utterance_order"])}
7
8      for score, hit in ranked_hits:
9          if not hit.payload: continue
10         doc_id = hit.payload.get('doc_id')
11         if not doc_id or doc_id in processed_ids: continue
12
13         idx = id_to_index.get(doc_id)
14         if idx is None: continue
15
16         # Window [idx-1, idx+1]
17         start = max(0, idx - 1)
18         end = min(len(self.kb_memory["utterance_order"]), idx
+ 2)
19
20         chunk_text = []
21         for i in range(start, end):
22             uid = self.kb_memory["utterance_order"][i]
23             processed_ids.add(uid)
24             u = self.kb_memory["utterances"][uid]
25             chunk_text.append(f"[{u['start']}] {u['speaker']}:
{u['text']}")
26

```

```

27         final_chunks.append("\n".join(chunk_text))
28
29     return final_chunks
30

```

4.3.3 Quy trình Truy xuất và Suy luận đa bước (Multi-step Retrieval and Reasoning)

Để giải quyết các truy vấn phức tạp đòi hỏi sự kết nối thông tin từ nhiều đoạn hội thoại khác nhau hoặc yêu cầu khả năng suy luận logic qua nhiều mốc thời gian, hệ thống được nâng cấp với quy trình truy xuất đa bước dựa trên tư tưởng của phương pháp IRCoT (Interleaving Retrieval with Chain-of-Thought) [17]. Điểm cốt lõi của quy trình này là sự tách biệt giữa luồng xử lý đơn giản và phức tạp thông qua một bộ định tuyến thông minh (*Query Router*).

Cơ chế định tuyến truy vấn (Query Routing)

Thay vì áp dụng một quy trình cố định cho mọi loại câu hỏi, hệ thống sử dụng lớp định tuyến (*Router Layer*) dựa trên kiến trúc đóng gói sẵn RouterQueryEngine (LlamaIndex).

LLM đóng vai trò là bộ chọn (*Selector*), phân tích đặc điểm của câu hỏi người dùng và lịch sử hội thoại để quyết định nhánh thực thi:

- **Nhánh đơn bước (Single-step Retrieval):** Áp dụng cho các truy vấn tra cứu sự thật trực tiếp (Fact Retrieval). Hệ thống sẽ đi thẳng vào quy trình tìm kiếm vector và trả lời như mô tả tại Mục 4.3.2.
- **Nhánh đa bước (Multi-step Reasoning):** Áp dụng cho các truy vấn đòi hỏi tổng hợp, so sánh hoặc truy vết chuỗi sự kiện phân tán. Nhánh này sẽ kích hoạt vòng lặp tương tác giữa suy luận và truy xuất.

Vòng lặp suy luận và truy xuất tương tác

Dựa trên triết lý lồng ghép của IRCoT [17], quy trình đa bước được thực hiện thông qua một chuỗi các bước lặp, trong đó mỗi bước bao gồm:

1. **Phân rã bài toán (Decomposition):** LLM phân tích câu hỏi phức tạp và xác định các thông tin thiếu hụt, từ đó tạo ra bước suy luận đầu tiên (*Thought*) và các câu hỏi phụ (*Sub-queries*).
2. **Truy xuất từng phần:** Với mỗi câu hỏi phụ, hệ thống gọi lại hàm `retrieve_context` hiện có để trích xuất các đoạn văn bản liên quan nhất.
3. **Tích hợp và Cập nhật tri thức:** Các thông tin mới tìm được (bao gồm cả ngữ cảnh mở rộng ± 1 lượt lời) được đưa vào bộ đệm ngữ cảnh (*Context Buffer*).
4. **Đánh giá và Tiếp nối:** LLM đánh giá tập tri thức hiện tại. Nếu thông tin vẫn chưa đủ để đưa ra kết luận cuối cùng, một bước suy luận mới sẽ được khởi tạo dựa trên các manh mối vừa tìm thấy, tạo thành một vòng lặp liên tục cho đến khi đạt được giới hạn số bước lặp hoặc thu thập đủ thông tin.

Tổng hợp phản hồi đa nguồn (Synthesis)

Sau khi kết thúc vòng lặp truy xuất, toàn bộ các đoạn hội thoại từ các bước lặp khác nhau sẽ được hợp nhất. Hệ thống sử dụng mô hình `bge-reranker` để đánh giá lại độ ưu tiên của toàn bộ kho ngữ cảnh vừa thu thập được nhằm loại bỏ các thông tin nhiễu phát sinh trong quá trình lặp.

Cuối cùng, mô hình **Gemini 2.5 Flash** sẽ thực hiện bước tạo sinh phản hồi cuối cùng, đảm bảo câu trả lời được neo giữ (*grounded*) chặt chẽ vào chuỗi dẫn chứng đa bước đã tìm được, giúp giải quyết triệt để hiện tượng ảo giác.

4.3.4 Tích hợp Mô hình ngôn ngữ và Tạo sinh (Generation)

Cơ chế tạo sinh đóng vai trò quan trọng trong việc sản xuất kết quả cuối cùng bằng cách tích hợp thông tin được truy xuất với truy vấn đầu vào. Sau khi thành phần truy xuất lấy kiến thức liên quan từ các nguồn bên ngoài, bộ tạo sinh tổng hợp thông tin này thành các phản hồi mạch lạc và phù hợp với ngữ cảnh. Mô hình Ngôn ngữ Lớn (LLM) đóng vai trò là xương sống của bộ tạo sinh, đảm bảo văn bản được tạo ra trôi chảy, chính xác và phù hợp với truy vấn gốc.

Lựa chọn Mô hình (Model Selection)

Hệ thống sử dụng *Google Gemini 2.5 Flash* làm mô hình nền tảng (*Foundation Model*). Quyết định này dựa trên báo cáo kỹ thuật của Google [15], trong đó nhấn mạnh ưu thế vượt trội của kiến trúc này về khả năng xử lý *Cửa sổ ngữ cảnh siêu lớn (Long Context Window)*. Khả năng này cho phép hệ thống nạp đồng thời nhiều đoạn hội thoại dài vào bộ nhớ đệm mà không làm mất mát thông tin, khắc phục hạn chế về độ dài đầu vào của các mô hình Transformer truyền thống.

Kỹ thuật Viết lại câu hỏi (Query Rewriting)

Một thách thức lớn trong hội thoại tự nhiên là sự xuất hiện của các đại từ thay thế (như "nó", "ông ấy") hoặc các câu hỏi tình lược. Để giải quyết vấn đề này, hệ thống áp dụng cơ chế viết lại câu hỏi trước khi thực hiện truy xuất nhằm đảm bảo tính đầy đủ của ngữ nghĩa. Cụ thể, hệ thống sẽ trích xuất lịch sử ba lượt hội thoại gần nhất và gửi đến mô hình ngôn ngữ lớn một yêu cầu chuyển đổi. Mô hình sẽ phân tích ngữ cảnh và thay thế các đại từ hoặc thành phần bị lược bỏ bằng các danh từ cụ thể đã xuất hiện trước đó. Câu hỏi sau khi được viết lại (*Rewritten Query*) mới được sử dụng làm đầu vào cho bước tìm kiếm vector, qua đó nâng cao đáng kể độ chính xác của kết quả truy xuất.

Chiến lược Prompting và Thiết kế Câu lệnh

Thay vì sử dụng phương pháp tinh chỉnh mô hình (*Fine-tuning*) vốn tốn kém tài nguyên, hệ thống áp dụng kỹ thuật *Học theo ngữ cảnh (In-context Learning)*. Theo khảo sát toàn diện của Liu và cộng sự [7], phương pháp này cho phép mô hình thích nghi nhanh với các tác vụ mới bằng cách quan sát các ví dụ hoặc dữ liệu được cung cấp ngay trong câu lệnh (Prompt) mà không cần cập nhật trọng số.

Khuôn mẫu câu lệnh (*Prompt Template*) cho bước tạo sinh cuối cùng được cấu trúc gồm 4 thành phần nhằm kiểm soát chặt chẽ chất lượng đầu ra và duy trì mạch hội thoại:

1. **Chỉ thị hệ thống (System Instruction):** Thiết lập vai trò trợ lý ảo và các quy tắc ứng xử (trung thực, ngắn gọn).
2. **Lịch sử trò chuyện (Chat History):** Cung cấp ngữ cảnh ngắn hạn của cuộc hội thoại hiện tại.

3. **Dữ liệu ngữ cảnh (Retrieved Context):** Các đoạn văn bản tìm được từ cơ sở dữ liệu vector, đóng vai trò là "nguồn sự thật" (*Ground Truth*).
4. **Câu hỏi người dùng (User Query):** Câu hỏi gốc của người dùng (hoặc câu đã viết lại).

Hiện thực kỹ thuật

Module được hiện thực hóa bằng Python thông qua thư viện `google-generativeai`. Mã nguồn dưới đây minh họa quá trình xây dựng prompt tổng hợp bao gồm cả lịch sử và ngữ cảnh:

```
1 def chat(self, user_query):
2     # 1. Xu ly lich su va viet lai cau hoi (Contextualizing)
3     if self.chat_history:
4         rewritten_query = self.rewrite_query(user_query)
5     else:
6         rewritten_query = user_query
7
8     # 2. Truy xuất thông tin (Retrieval)
9     context_chunks = self.retrieve_context(rewritten_query)
10    final_context_str = "\n---\n".join(context_chunks)
11
12    # 3. Xây dựng Prompt tổng hợp
13    # G ghép lịch sử chat 3 lượt gần nhất
14    history_str = format_history(self.chat_history[-3:])
15
16    prompt = f"""
17    Bạn là trợ lý ảo chuyên nghiệp.
18    QUY TẮC:
19    1. Chỉ trả lời dựa trên [THÔNG TIN ĐƯỢC CUNG CẤP].
20    2. Nếu thông tin không đủ, hãy nói "Tai lieu khong de cap".
21
22    --- LỊCH SỬ TRỢ CHUYỀN ---
23    {history_str}
24
25    --- THÔNG TIN ĐƯỢC CUNG CẤP (Context) ---
26    {final_context_str}
```

```

27
28     --- CAU HOI ---
29     User: {user_query}
30
31     Cau tra loi:
32     """
33
34     # 4. Goi API Gemini
35     try:
36         response = llm_model.generate_content(prompt)
37         # Luu lich su cho luot sau
38         self.chat_history.append((user_query, response.text))
39         return response.text
40     except Exception as e:
41         return "Loi he thong."

```

Listing 4.3: Hàm tạo sinh câu trả lời có ngữ cảnh lịch sử

Việc đưa trực tiếp lịch sử trò chuyện vào prompt giúp mô hình Gemini hiểu được ngữ cảnh liên tục, tạo ra trải nghiệm người dùng mượt mà và tự nhiên hơn so với các hệ thống hỏi đáp tĩnh (stateless).

Chapter 5

Đánh Giá

5.1 Đánh giá Hệ thống

Chương này trình bày quá trình đánh giá toàn diện hệ thống theo từng thành phần trong pipeline, từ xử lý tín hiệu tiếng nói đến truy xuất và tổng hợp thông tin. Mục tiêu của việc đánh giá là kiểm chứng tính chính xác, độ ổn định và khả năng hoạt động trong điều kiện dữ liệu thực tế có nhiễu.

Do hệ thống được xây dựng theo kiến trúc nhiều tầng, quá trình đánh giá được tổ chức theo hướng phân rã theo từng mô-đun, kết hợp với các chỉ số định lượng và các kịch bản thực nghiệm.

5.1.1 Dữ liệu đánh giá (Evaluation Dataset)

Hệ thống được đánh giá trên hai loại dữ liệu nhằm đảm bảo cả tính chuẩn hóa và tính thực tế.

(1) AMI Corpus (Annotated Dataset): Được sử dụng để đánh giá bài toán định danh người nói (speaker diarization). Dữ liệu có nhãn người nói theo thời gian, cho phép đo chính xác các chỉ số như DER và SAC.

(2) Dữ liệu hội thoại thực tế (BIWASE): Bao gồm 04 bản ghi âm (khoảng 30 phút) được xử lý qua ASR.

Đặc điểm:

- Không có dấu câu, chứa lỗi nhận dạng (ASR noise)

- Thông tin phân tán, nhiều số liệu và thực thể chuyên ngành

(3) Tập Q&A tự xây dựng:

- Quy mô: 50 cặp câu hỏi – đáp án
- Nguồn: 5 bản transcript
- Mỗi mẫu gồm:
 - Query
 - Ground Truth Answer
 - Reference Context ID

5.1.2 Đánh giá Định danh người nói

Các thí nghiệm đánh giá diarization được thực hiện trên cùng một tập dữ liệu, với cùng cấu hình tiền xử lý và VAD. Mỗi cấu hình chỉ thay đổi một thành phần tại một thời điểm nhằm đảm bảo tính công bằng trong so sánh.

Chất lượng diarization được đánh giá bằng hai chỉ số: Diarization Error Rate (DER) và chất lượng ánh xạ nhãn (label matching quality), trong đó DER phản ánh lỗi tổng thể ở mức thời gian, còn label matching quality phản ánh mức độ ổn định của cụm người nói.

Chỉ số DER tổng hợp phản ánh chất lượng phân đoạn tổng thể, tuy nhiên các thành phần cấu thành như Miss và Confusion có ý nghĩa quan trọng trong bối cảnh hệ thống xử lý tiếng nói nhiều tầng. Đặc biệt, lỗi Miss ảnh hưởng trực tiếp đến lượng dữ liệu đầu vào cho ASR, trong khi lỗi Confusion là chỉ số trực tiếp để so sánh độ hiệu quả của các mô hình.

Kịch bản 1: So sánh giữa ECAPA-TDNN và TitaNet-large

Kịch bản này nhằm so sánh hai mô hình trích xuất speaker embedding phổ biến là ECAPA-TDNN và TitaNet trong cùng điều kiện dữ liệu và siêu tham số. Các mô hình được đánh giá trực tiếp thông qua kết quả diarization tổng thể, thay vì hiệu suất embedding riêng lẻ. Mục tiêu là xác định mô hình cho kết quả ổn định hơn và phù hợp hơn với pipeline hiện tại.

Table 5.1: So sánh ECAPA-TDNN và TitaNet trên các chỉ số diarization

Mô hình	DER	Miss	Conf	Quality
ECAPA-TDNN (Set 1)	0.49	0.13	0.31	0.54
TitaNet-Large (Set 1)	0.44	0.13	0.26	0.59
ECAPA-TDNN (Set 2)	0.54	0.08	0.38	0.53
TitaNet-Large (Set 2)	0.53	0.08	0.37	0.54

Nhận xét: Kết quả cho thấy hai kiến trúc ECAPA-TDNN và TitaNet-Large không có khác biệt đáng kể về DER và Quality tổng thể. Với cùng các bộ siêu tham số, Miss không thay đổi, Confusion Error của TitaNet-Large cho kết quả thấp hơn.

Giữa hai tập siêu tham số, Set 2 cho tỷ lệ Miss thấp hơn đáng kể so với Set 1, tuy nhiên lại làm tăng lỗi Confusion, dẫn đến DER tổng thể cao hơn. Kết quả này phản ánh sự đánh đổi giữa việc giảm Miss và khả năng phân cụm chính xác các người nói trong pipeline diarization.

Kết luận: Hai mô hình cho kết quả tương đối gần nhau trong các cấu hình thử nghiệm. Trong bối cảnh pipeline hiện tại, sự khác biệt giữa hai mô hình không mang ý nghĩa thống kê rõ ràng, và việc lựa chọn mô hình phụ thuộc nhiều hơn vào độ ổn định, thời gian xử lý và sự phù hợp với các bước xử lý phía sau như ASR.

Kịch bản 2: So sánh giữa Spectral Clustering và AHC

Kịch bản này đánh giá ảnh hưởng của thuật toán phân cụm đến chất lượng diarization, khi giữ cố định mô hình embedding. Hai phương pháp phân cụm được so sánh là Spectral Clustering và Agglomerative Hierarchical Clustering (AHC), với mục tiêu lựa chọn phương án cho kết quả nhất quán và ít lỗi hơn trong pipeline.

Kết quả thực nghiệm cho thấy Spectral Clustering đạt chất lượng tổng thể cao hơn so với AHC. Cụ thể, giá trị Quality trung bình trên 16 phiên hội họp của Spectral đạt 0.7075, cao hơn so với 0.6913 của AHC. Đồng thời, Spectral Clustering cho thấy tỷ lệ Miss thấp hơn (0.088 so với 0.099), cho thấy khả năng giữ lại nội dung thoại tốt hơn, qua đó giảm nguy cơ mất thông tin quan trọng cho bước nhận dạng tiếng nói. Tỷ lệ Confusion giữa hai phương pháp là tương đương, tuy nhiên Spectral vẫn duy trì lợi thế nhỏ về độ ổn định khi số lượng người nói tăng.

Từ các kết quả trên, Spectral Clustering được lựa chọn làm phương pháp phân cụm mặc định trong pipeline diarization của hệ thống, trong khi AHC được sử dụng như một phương pháp baseline để đối chứng.

Kịch bản 3: So sánh 1 lớp phân tích và 2 lớp phân tích

So sánh giữa cấu hình Spectral Clustering một lớp và hai lớp cho thấy cấu hình hai lớp đạt hiệu năng tốt hơn về mặt chất lượng phân cụm, đặc biệt khi xét trên các chỉ số phản ánh trực tiếp độ chính xác của diarization pipeline. Cụ thể, cấu hình một lớp đạt DER xấp xỉ 0.10, với Miss khoảng 0.08 và Confusion quanh mức 0.20, trong khi cấu hình hai lớp đạt DER khoảng 0.097, với Confusion giảm xuống còn khoảng 0.18.

Mặc dù mức cải thiện DER tổng thể là không lớn, sự khác biệt thể hiện rõ hơn ở chỉ số Quality. Cấu hình một lớp đạt Quality trung bình khoảng 0.70, trong khi cấu hình hai lớp đạt khoảng 0.72. Cần lưu ý rằng Quality không phải là thang đo tuyến tính, do đó mức tăng từ 0.70 lên 0.72 phản ánh sự cải thiện đáng kể về độ nhất quán nhãn và mức độ khớp giữa các cụm người nói, đặc biệt trong các vùng ranh giới khó và các đoạn hội thoại có chồng lấn.

Kết quả này cho thấy lớp phân tích bổ sung giúp giảm đáng kể lỗi Confusion, vốn là thành phần lỗi ảnh hưởng trực tiếp đến chất lượng diarization và các bước xử lý phía sau như ASR và truy xuất nội dung. Do đó, dù các phân đoạn ngắn chỉ chiếm tỷ lệ nhỏ trong tổng thời lượng, tác động của cấu hình hai lớp vẫn thể hiện rõ ràng trên các chỉ số chất lượng cốt lõi.

So với các kết quả được báo cáo trong các công trình trước, bao gồm các hệ thống dựa trên *pyannote* và các bộ benchmark tổng hợp như *SDBench*[10], mức DER đạt được trong kịch bản này nằm trong cùng dải giá trị với các hệ thống diarization hiện đại trong điều kiện dữ liệu tương tự. Điều này cho thấy pipeline đề xuất đạt được hiệu năng cạnh tranh, đồng thời duy trì tính đơn giản và khả năng kiểm soát trong thiết kế.

Từ góc độ hệ thống, cấu hình hai lớp được xem là phương án phù hợp khi mục tiêu ưu tiên giảm Confusion và cải thiện chất lượng nhãn người nói, đặc biệt trong bối cảnh xử lý cuộc họp nhiều người và phục vụ cho các tác vụ truy xuất và tổng hợp thông tin ở giai đoạn sau.

5.1.3 Đánh giá tổng quan pipeline chuyển đổi âm thanh thành văn bản

Vai trò: Pipeline chuyển đổi âm thanh thành văn bản là thành phần đầu vào của toàn bộ hệ thống. Chất lượng của bước này ảnh hưởng trực tiếp đến các mô-đun phía sau như summarization, retrieval và question answering. Nếu transcript bị mất nội dung hoặc gán sai speaker, các lỗi này sẽ tiếp tục lan truyền sang các tầng xử lý downstream và làm giảm chất lượng của toàn bộ hệ thống. Vì vậy, quá trình đánh giá pipeline tập trung vào khả năng bảo toàn thông tin hội thoại cũng như độ chính xác của speaker metadata trong transcript đầu ra. **Tiêu chuẩn đánh giá:** Quá trình đánh giá được thực hiện dựa trên ba tiêu chí chính gồm khả năng bảo toàn nội dung hội thoại, độ chính xác của speaker attribution và tính nhất quán của transcript theo thời gian.

- **Tính toàn vẹn nội dung:** nội dung hội thoại không bị mất trong quá trình speech recognition và diarization.
- **Độ chính xác định danh người nói:** các lượt phát biểu được gán đúng speaker tương ứng.
- **Tính nhất quán theo thời gian:** thứ tự và mốc thời gian của các phát biểu được duy trì ổn định trong transcript đầu ra.

Chỉ số đo lường: Hệ thống sử dụng kết hợp DER và SAC để đánh giá chất lượng tổng thể của pipeline chuyển đổi âm thanh thành văn bản.

- **DER:** đánh giá lỗi diarization theo timeline của audio, tập trung chủ yếu vào hai thành phần Miss và Confusion.
- **SAC:** đánh giá đồng thời khả năng bảo toàn nội dung và độ chính xác của speaker attribution trong transcript cuối cùng.

Khác với DER chỉ phản ánh mức độ sai lệch của speaker segmentation theo thời gian, SAC phản ánh trực tiếp hơn chất lượng của transcript ở mức nội dung và ngữ nghĩa. Điều này đặc biệt quan trọng đối với các tác vụ downstream như retrieval theo participant hoặc meeting summarization.

Kết quả: Kết quả thực nghiệm trên tập dữ liệu AMI cho thấy pipeline đạt khả năng bảo toàn nội dung tương đối ổn định trên nhiều cuộc họp có thời lượng và số lượng speaker khác nhau. Chỉ số Coverage đạt trung bình khoảng 0.8456 ± 0.0764 , cho thấy phần lớn nội dung hội thoại đã được giữ lại sau quá trình xử lý audio và diarization. Trong khi đó, Speaker Correctness đạt trung bình khoảng 0.7552 ± 0.0726 , phản ánh khả năng duy trì tương đối tốt mối liên hệ giữa nội dung phát biểu và speaker metadata. Từ hai thành phần trên, hệ thống đạt giá trị SAC trung bình khoảng 0.6434 ± 0.1080 . Kết quả này cho thấy transcript đầu ra vẫn duy trì được phần lớn nội dung hội thoại đồng thời giữ được mức độ chính xác tương đối đối với speaker attribution. Tuy nhiên, chất lượng speaker assignment vẫn giảm trong các cuộc họp có nhiều người tham gia hoặc có hiện tượng chồng lấn lời nói.

Thống kê thực nghiệm:

Chỉ số	Trung bình
Coverage	0.8456 ± 0.0764
Speaker Correctness	0.7552 ± 0.0726
SAC	0.6434 ± 0.1080

Table 5.2: Kết quả đánh giá pipeline chuyển đổi âm thanh thành văn bản

Nhận xét:

Kết quả thực nghiệm cho thấy pipeline có khả năng duy trì phần lớn nội dung hội thoại trong quá trình chuyển đổi từ audio sang transcript. Giá trị Coverage ở mức cao cho thấy hệ thống hạn chế được việc mất thông tin quan trọng trong quá trình speech recognition và diarization. Tuy nhiên, Speaker Correctness thấp hơn Coverage cho thấy việc gán đúng nội dung cho đúng speaker vẫn là một bài toán khó, đặc biệt trong các cuộc họp có nhiều speaker hoặc xuất hiện hiện tượng chuyển lượt lời liên tục. Điều này phản ánh rằng dù transcript vẫn giữ được nội dung hội thoại tương đối đầy đủ, các lỗi speaker attribution vẫn có thể làm giảm chất lượng của các tác vụ downstream. So với Miss, lỗi Confusion có ảnh hưởng nghiêm trọng hơn đối với hệ thống retrieval và summarization đa tầng. Việc gán sai speaker có thể làm sai lệch quá trình truy xuất theo participant, phân tích đóng góp cá nhân hoặc tổng hợp thông tin cuộc họp. Điều này cho thấy việc

tối ưu diarization không chỉ dừng ở giảm DER tổng thể mà cần tập trung nhiều hơn vào việc duy trì tính nhất quán giữa nội dung phát biểu và speaker metadata trong transcript cuối cùng.

5.1.4 Đánh giá tóm tắt đa tầng

Vai trò: Mô-đun tóm tắt đa tầng đóng vai trò giảm kích thước dữ liệu hội thoại đầu vào đồng thời hỗ trợ quá trình retrieval trên các transcript dài và nhiều chủ đề. Thay vì truy xuất trực tiếp trên toàn bộ nội dung cuộc họp, hệ thống sử dụng các tầng tóm tắt trung gian để biểu diễn thông tin ở nhiều mức độ ngữ nghĩa khác nhau như participant-level, topic-level và meeting-level.

Việc đánh giá mô-đun này tập trung vào khả năng duy trì các thông tin quan trọng của cuộc họp trong khi vẫn đảm bảo mức độ cô đọng phù hợp cho indexing và retrieval. Ngoài ra, hệ thống cũng cần hạn chế việc sinh ra các thông tin không tồn tại trong transcript gốc nhằm đảm bảo độ tin cậy của dữ liệu tóm tắt.

Tiêu chuẩn đánh giá: Quá trình đánh giá được thực hiện dựa trên ba tiêu chí chính gồm khả năng bảo toàn thông tin, khả năng nén dữ liệu và mức độ bám sát transcript gốc.

- **Tính đầy đủ thông tin (Coverage):** bản tóm tắt cần giữ lại phần lớn các information units quan trọng của cuộc họp.
- **Độ súc tích (Compression):** hệ thống cần giảm đáng kể kích thước dữ liệu đầu vào nhưng vẫn duy trì được khả năng biểu diễn nội dung chính.
- **Tính xác thực (Faithfulness):** nội dung tóm tắt không được sinh thêm các thông tin ngoài ngữ cảnh transcript ban đầu.

Chỉ số đo lường:

Hệ thống sử dụng ba chỉ số chính gồm Information Coverage Score (ICS), Compression Ratio (CR) và Hallucination Rate (HR) để đánh giá chất lượng của mô-đun summarization đa tầng.

- **Information Coverage Score (ICS):** đo mức độ bảo toàn thông tin quan trọng giữa ground truth và bản tóm tắt sinh ra thông qua semantic similarity và lexical overlap.

- **Compression Ratio (CR):** phản ánh tỷ lệ giữa độ dài của bản tóm tắt và transcript gốc.
- **Hallucination Rate (HR):** đo mức độ xuất hiện của các thông tin không tồn tại trong transcript đầu vào.

Khác với các phương pháp chỉ đánh giá độ tương đồng bề mặt giữa văn bản gốc và bản tóm tắt, các chỉ số trên tập trung nhiều hơn vào khả năng bảo toàn information units phục vụ retrieval và indexing ở nhiều tầng ngữ nghĩa khác nhau.

Kết quả: Kết quả thực nghiệm cho thấy hệ thống tóm tắt đa tầng có khả năng duy trì tương đối tốt các nội dung quan trọng của cuộc họp trong khi vẫn giảm mạnh kích thước dữ liệu cần xử lý.

Chỉ số Information Coverage Score đạt trung bình khoảng 0.3310 ± 0.0425 , phản ánh khả năng giữ lại các information units quan trọng ở nhiều tầng summarization khác nhau. Mặc dù giá trị coverage không quá cao, kết quả này phù hợp với đặc thù của bài toán abstractive summarization đa tầng, nơi nội dung thường được rút gọn và diễn đạt lại thay vì giữ nguyên cấu trúc văn bản ban đầu.

Compression Ratio đạt khoảng 0.0653 ± 0.0287 , cho thấy hệ thống đã giảm đáng kể kích thước transcript đầu vào. Điều này giúp giảm chi phí indexing và cải thiện tốc độ retrieval trên tập dữ liệu nhiều cuộc họp.

Hallucination Rate được duy trì ở mức thấp khoảng 0.0910 ± 0.0879 , cho thấy phần lớn nội dung sinh ra vẫn bám sát transcript gốc và ít xuất hiện các thông tin ngoài ngữ cảnh.

Thống kê thực nghiệm:

Chỉ số	Trung bình
ICS	0.3510 ± 0.0625
CR	0.0653 ± 0.0287
HR	0.0910 ± 0.0879

Table 5.3: Kết quả đánh giá hệ thống tóm tắt đa tầng

Nhận xét:

Kết quả thực nghiệm cho thấy hệ thống tóm tắt đa tầng có khả năng giảm mạnh kích thước dữ liệu hội thoại trong khi vẫn duy trì được phần lớn các nội dung quan trọng phục vụ retrieval. Compression Ratio thấp cho thấy transcript đã được rút gọn đáng kể, từ đó giúp giảm chi phí indexing và tăng tốc độ truy xuất trên tập dữ liệu lớn.

Bên cạnh đó, Hallucination Rate được duy trì ở mức thấp chứng tỏ phần lớn nội dung sinh ra vẫn bám sát transcript gốc và có độ tin cậy phù hợp để sử dụng như lớp biểu diễn trung gian trong hệ thống retrieval đa tầng.

Tuy nhiên, giá trị ICS chưa quá cao cho thấy quá trình nén thông tin vẫn làm mất một phần information units, đặc biệt đối với các chi tiết nhỏ, nội dung mang tính định lượng hoặc các thông tin phụ xuất hiện rải rác trong cuộc họp. Điều này phản ánh sự đánh đổi giữa khả năng giảm kích thước dữ liệu và mức độ bảo toàn thông tin trong bài toán summarization đa tầng.

Ngoài ra, kết quả đo lường ICS cũng bị ảnh hưởng một phần bởi đặc điểm của bộ ground truth. Các ground truth summaries trong AMI được xây dựng từ nhiều đoạn hội thoại và annotation khác nhau, nên nội dung thường chưa hoàn toàn cô đọng và vẫn tồn tại một lượng nhiễu nhất định. Một số information units mang tính lặp lại, rời rạc hoặc ít giá trị ngữ nghĩa vẫn xuất hiện trong dữ liệu tham chiếu. Trong phạm vi nghiên cứu, hệ thống chưa áp dụng các cơ chế lọc nhiễu hoặc chuẩn hóa information units ở mức sâu trước khi đánh giá, do đó kết quả coverage có thể bị giảm trong một số trường hợp dù nội dung tóm tắt sinh ra vẫn giữ được các ý chính quan trọng của cuộc họp.

5.1.5 So sánh hiệu năng truy xuất (Advanced RAG và Naive RAG)

Vai trò: Mô-đun truy xuất đóng vai trò quyết định trong việc thiết lập tập ngữ cảnh đầu vào cho mô hình sinh. Đánh giá này nhằm kiểm chứng sự vượt trội của kiến trúc RAG nâng cao (Advanced RAG) do đề án đề xuất so với phương pháp RAG truyền thống (Naive RAG) trong việc định vị, bao phủ và ưu tiên các thông tin trọng yếu.

Thiết lập so sánh:

Naive RAG (Baseline): Sử dụng chiến lược phân mảnh (*chunking*) truyền thống, trong đó mỗi câu thoại được xem là một đơn vị ngữ nghĩa độc lập.

Advanced RAG (Hệ thống đề xuất): Vận hành dựa trên quy trình tối ưu hóa đa tầng

tích hợp các kỹ thuật: (i) Viết lại câu hỏi (*Query Rewriting*) ở giai đoạn tiền truy xuất để làm rõ ngữ nghĩa dựa trên lịch sử hội thoại; (ii) Chiến lược *Parent Document Retrieval* (băm nhỏ câu thoại thành các *chunk* 40 từ với độ chồng lấp 10 từ) ở giai đoạn truy xuất để tăng mật độ vector; và (iii) Kết hợp bộ tái xếp hạng (*Cross-Encoder*) cùng cơ chế Cửa sổ trượt (*Sliding Window*) ở giai đoạn hậu truy xuất nhằm tinh lọc kết quả và khôi phục đầy đủ ngữ cảnh hội thoại.

Tiêu chuẩn và Chỉ số đo lường: Quá trình đánh giá tập trung vào khả năng tìm đúng tài liệu (**Hit Rate@K**), khả năng xếp hạng văn bản (**MRR**), và khả năng cô đọng thông tin (**Context Relevance**).

Kết quả thực nghiệm:

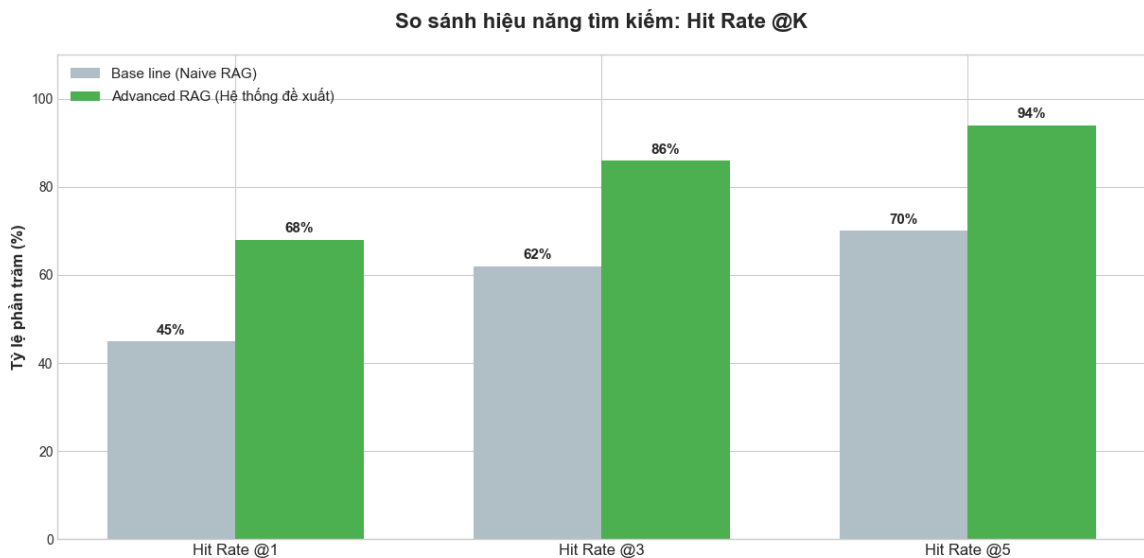


Figure 5.1: So sánh hiệu năng truy xuất (Hit Rate @K) giữa phương pháp phân mảnh nguyên câu (Naive RAG) và hệ thống đề xuất với chiến lược Parent Document Retrieval (Advanced RAG)

Dựa trên tập dữ liệu đánh giá 50 truy vấn (Q&A pairs), kiến trúc Advanced RAG cho thấy sự cải thiện toàn diện:

Về Hit Rate@K: Hệ thống Advanced RAG đạt Hit Rate@1 là 68% và Hit Rate@5 lên tới 94%, cải thiện vượt bậc so với mức 45% và 70% của cấu hình Naive RAG.

Về MRR: Điểm MRR của hệ thống đề xuất dao động dao động trong khoảng từ 0.756 đến 0.790, minh chứng cho việc các đoạn hội thoại chứa đáp án chính xác luôn được

bộ tái xếp hạng ưu tiên đẩy lên các vị trí Top đầu.

Về Context Relevance: Do hiệu ứng mở rộng của Cửa sổ trượt, điểm Context Relevance của Advanced RAG có sự sụt giảm nhẹ so với Naive RAG do hệ thống chủ động thu thập thêm các câu thoại lân cận.

Nhận xét: Sự gia tăng đột phá của chỉ số Hit Rate khẳng định hiệu quả cộng hưởng từ chuỗi tối ưu hóa toàn diện của hệ thống Advanced RAG. Trong đó, việc viết lại câu truy vấn giúp xác định chính xác mục tiêu tìm kiếm, chiến lược *Parent Document* giúp giải quyết triệt để điểm yếu cố hữu của Naive RAG là hiện tượng phân mảnh ngữ nghĩa (*semantic fragmentation*). Đặc biệt, sự can thiệp của bộ tái xếp hạng (*Cross-Encoder*) đóng vai trò quyết định trong việc sàng lọc nhiễu và đẩy các dẫn chứng thực sự liên quan lên vị trí ưu tiên, giúp tối ưu hóa chỉ số Hit Rate@1 và MRR. Mặc dù có sự đánh đổi nhỏ về Context Relevance do thu thập thêm một lượng hội thoại phụ trợ xung quanh, đây là sự hy sinh có chủ đích và tối ưu. Lượng thông tin mở rộng từ cơ chế cửa sổ trượt cung cấp đủ luồng giao tiếp (*contextual flow*) làm cơ sở để Mô hình Ngôn ngữ Lớn (LLM) suy luận chính xác đối với các truy vấn phức tạp, loại bỏ hoàn toàn tình trạng đứt gãy thông tin khi chỉ truy xuất các câu thoại đơn lẻ.

5.1.6 Đánh giá hiệu quả của siêu dữ liệu (Metadata Filtering)

Vai trò: Trong môi trường hội thoại đa diễn giả, việc chỉ dựa vào độ tương đồng ngữ nghĩa (Semantic Search) thuần túy thường dẫn đến sai lệch khi nhiều người cùng thảo luận về một chủ đề bằng các hệ thống từ vựng giống nhau. Siêu dữ liệu (định danh người nói, mốc thời gian) được hệ thống lưu trữ trực tiếp dưới dạng *payload* trong cơ sở dữ liệu vector đóng vai trò như một bộ lọc cứng (*hard filter*). Đánh giá này nhằm xác định mức độ đóng góp của siêu dữ liệu trong việc thu hẹp không gian tìm kiếm và loại bỏ triệt để hiện tượng nhầm lẫn nguồn phát biểu.

Thiết lập so sánh:

Cấu hình cơ sở (Không có Metadata): Truy xuất tài liệu hoàn toàn dựa trên khoảng cách vector ngữ nghĩa.

Cấu hình đề xuất (Có Metadata Filtering): Kết hợp tìm kiếm vector với kỹ thuật lọc

siêu dữ liệu (khoanh vùng theo *Speaker* hoặc khoảng thời gian) trước khi tiến hành tính điểm tương đồng.

Tiêu chuẩn và Chỉ số đo lường:

Khả năng phân luồng và cải thiện độ chính xác truy xuất (đo lường bằng chỉ số **Hit Rate@K**).

Khả năng đẩy các đoạn hội thoại chứa đáp án của đúng người phát biểu lên vị trí ưu tiên cao nhất (đo lường bằng chỉ số **MRR**).

Kết quả thực nghiệm: Trên các tập truy vấn đặc thù yêu cầu xác định đích danh chủ thể hành động hoặc thời điểm đưa ra quyết định (ví dụ: "Ai là người đề xuất thêm màn hình hiển thị pin và lo ngại về chi phí?"), việc tích hợp bộ lọc metadata mang lại hiệu quả vượt trội. Cụ thể, hệ thống ghi nhận chỉ số Hit Rate@5 tăng từ 82% lên 94%, đồng thời MRR cải thiện từ 0.68 lên 0.76.

Thực nghiệm cho thấy trong tệp *final_transcriptions1.json*, khi cả Speaker 1 và Speaker 0 cùng thảo luận về "battery status", hệ thống Naive RAG thường nhầm lẫn giữa người đưa ra ý tưởng (*Speaker 1*: "handy to put a battery status display on it") và người hỏi lại ngữ cảnh (*Speaker 0*: "How much is left in the battery?"). Bằng cách áp dụng bộ lọc *Speaker ID*, hệ thống Advanced RAG đã cô lập chính xác các phát biểu của Speaker 1, từ đó xác định đúng nhân vật này vừa là người đề xuất vừa là người lo ngại về chi phí sản xuất (*drag up the production costs*).

Nhận xét: Sự gia tăng đồng thời của cả hai chỉ số khẳng định vai trò then chốt của siêu dữ liệu trong việc giải quyết vấn đề chồng lấp từ vựng. Cơ chế *Metadata Filtering* giúp hệ thống bóc tách chính xác các thực thể trong môi trường hội thoại đa người nói vốn có mật độ từ khóa tương đồng rất cao.

Điều này đặc biệt mang ý nghĩa quyết định đối với các truy vấn về mốc thời gian và phân công nhiệm vụ. Ví dụ, tại tệp *final_transcriptions4.json*, nếu chỉ tìm kiếm từ khóa "minutes", hệ thống có thể truy xuất nhầm các lượt thảo luận về thời gian họp ở đầu buổi. Tuy nhiên, nhờ siêu dữ liệu thời gian (*start_time*: 00:34:41), hệ thống đã lọc chính xác lượt phát biểu cuối cuộc họp của Speaker 0 về việc lưu trữ biên bản vào thư mục dự án và giao nhiệm vụ cho nhà thiết kế công nghiệp. Nhờ siêu dữ liệu, Mô hình Ngôn ngữ Lớn (LLM) không còn bị "mù" thông tin chủ thể, đảm bảo câu trả lời luôn

được gắn đúng với diễn giả và mốc thời gian thực tế của sự kiện.

5.1.7 Đánh giá khả năng phục hồi tri thức và chịu lỗi của hệ thống RAG

Vai trò và Mục tiêu: Trong môi trường vận hành thực tế, dữ liệu đầu vào từ mô-đun nhận dạng tiếng nói (ASR) không thể tránh khỏi các sai số, đặc biệt là lỗi đồng âm hoặc nhận diện sai thuật ngữ chuyên ngành. Đánh giá này nhằm kiểm chứng độ bền vững (*Robustness*) và khả năng chịu lỗi (*Fault Tolerance*) của pipeline RAG; cụ thể là năng lực tự phục hồi ngữ nghĩa để đảm bảo trích xuất đúng ngữ cảnh và sinh ra câu trả lời chính xác từ một kho văn bản chứa nhiễu.

Tiêu chuẩn đánh giá:

- **Khả năng ánh xạ ngữ nghĩa:** Đánh giá việc hệ thống có "hiểu" đúng ý định truy vấn khi đối chiếu với các từ khóa bị biến dạng trong văn bản gốc hay không.
- **Khả năng truy xuất chịu lỗi:** Năng lực định vị và truy xuất đúng đoạn ngữ cảnh bất chấp sự sai lệch về mặt từ vựng (*Lexical Mismatch*).
- **Khả năng chuẩn hóa tạo sinh:** Mức độ LLM tự động hiệu chỉnh, chuẩn hóa các thuật ngữ bị lỗi ASR thành từ vựng chính xác trong câu trả lời cuối cùng.

Chỉ số đo lường:

- **Đánh giá định lượng:** Thông qua sự suy giảm của chỉ số Hit Rate@K khi vận hành trên tập dữ liệu chứa nhiễu (ASR Transcripts) so với kỳ vọng lý thuyết.
- **Đánh giá định tính:** Thông qua việc kiểm tra thủ công tính toàn vẹn và logic của câu trả lời trên các kịch bản đặc thù.

Kết quả thực nghiệm: Thực nghiệm cho thấy hệ thống duy trì được mức hiệu năng ấn tượng với chỉ số Hit Rate@5 đạt **89%** ngay cả khi dữ liệu đầu vào chứa tỷ lệ lỗi ASR đáng kể. Phân tích định tính trên tập dữ liệu dự án thực tế (BIWASE) cho thấy hệ thống xử lý thành công các trường hợp lỗi nhận dạng đặc thù. Điển hình, các lỗi phiên âm như "ô đê a" được hệ thống ngầm hiểu và ánh xạ chính xác tới khái niệm "ODA", hay thuật

ngữ “bi hoa xe” được truy xuất đúng về cụm từ “BIWASE” trong phần lớn các truy vấn thử nghiệm.

Nhận xét và Thảo luận: Khả năng ”tự sửa lỗi” và phục hồi tri thức này xuất phát từ sức mạnh tổng hợp của kiến trúc hệ thống:

1. **Ở khâu truy xuất:** Cơ chế tìm kiếm ngữ nghĩa (*Semantic Search*) kết hợp bộ tái xếp hạng chéo (*Cross-Encoder*) giúp đánh giá độ tương đồng dựa trên ngữ cảnh đa chiều. Hệ thống nắm bắt ý nghĩa tổng thể của đoạn thoại thay vì phụ thuộc vào việc so khớp chính xác từng ký tự (*Keyword Matching*), qua đó vô hiệu hóa các nhiễu từ vựng cục bộ.
2. **Ở khâu tạo sinh:** Năng lực ngôn ngữ của LLM (*Parametric Knowledge*) đóng vai trò như một bộ lọc chuẩn hóa. Khi tiếp nhận ngữ cảnh có chứa lỗi chính tả, mô hình dựa vào tư duy logic để tự động hiệu chỉnh và sinh ra câu trả lời với thuật ngữ chuyên ngành chuẩn xác.

Tuy nhiên, kiến trúc này vẫn tồn tại những giới hạn nhất định. Trong các tình huống lỗi ASR làm thay đổi hoàn toàn hoặc đảo lộn nghiêm trọng cấu trúc ngữ nghĩa của câu (khiến vector bị dịch chuyển quá xa trong không gian nhúng), hệ thống vẫn có nguy cơ truy xuất sai ngữ cảnh, dẫn đến hiện tượng thiếu hụt thông tin hoặc phản hồi không như mong muốn.

Chapter 6

Kết Luận

6.1 Kết luận

Đồ án này đã nghiên cứu và đề xuất giải pháp toàn diện cho bài toán khai phá tri thức từ dữ liệu hội thoại đa người nói. Nhận thức rõ những giới hạn công nghệ hiện tại của các mô hình nhận dạng âm thanh, nghiên cứu đặt trọng tâm vào việc phát triển hệ thống truy xuất thông tin nâng cao (Advanced RAG) và ứng dụng Mô hình Ngôn ngữ Lớn (LLM) để hậu xử lý, qua đó đạt được những đóng góp chính sau:

Thiết lập pipeline xử lý tiếng nói cơ sở: Đồ án đã tích hợp thành công các mô hình Phân đoạn người nói (Speaker Diarization) và Nhận dạng tiếng nói (ASR) tạo thành một đường ống tiền xử lý đầu cuối. Mặc dù đóng vai trò nền tảng trong việc số hóa dữ liệu, các mô-đun này thực tế đã chạm ngưỡng giới hạn hiệu năng khi đối mặt với dữ liệu hội thoại mang nhiều nhiễu, chồng lấn lời nói và không thể đảm bảo tính toàn vẹn tuyệt đối về định danh hay thời gian. Sự thiếu hoàn hảo này chính là tiền đề thiết yếu để hệ thống phát triển các cơ chế dung lỗi ở tầng NLP phía sau.

Phát triển kiến trúc Advanced RAG chịu lỗi và LLM Correction: Để khắc phục triệt để sai sót tích lũy từ mô-đun ASR và Diarization, đồ án đã thiết kế kiến trúc RAG nâng cao kết hợp cơ chế hiệu chỉnh ngữ nghĩa của LLM (LLM Correction). Bằng cách tối ưu hóa kỹ thuật phân mảnh văn bản (*chunking*) và chiến lược tái xếp hạng (*Re-ranking*), hệ thống có khả năng dung lỗi (*fault tolerance*) cao, tự động phục hồi và chuẩn hóa thông tin từ khóa thay vì phụ thuộc vào một bản ghi (transcript) hoàn hảo.

Hiện thực hóa cơ chế Truy xuất và Suy luận đa bước (Multi-step Reasoning): Vượt qua giới hạn của hệ thống hỏi đáp thông thường, đề án đã xây dựng khung giải pháp tích hợp vòng lặp truy xuất - suy luận tương tác. Cơ chế này cho phép hệ thống tự động bóc tách các câu hỏi phức tạp (multi-hop queries), truy vết chuỗi sự kiện bị phân mảnh qua nhiều mốc thời gian và cung cấp đáp án chính xác kèm dẫn chứng minh bạch.

Xây dựng chiến lược Tóm tắt đa tầng (Multi-level Summarization): Hệ thống phát triển năng lực tổng hợp tri thức theo cấu trúc phân cấp linh hoạt: từ tóm tắt tổng quan toàn bộ cuộc họp, đến tóm tắt chi tiết theo chủ đề thảo luận và tóm tắt theo từng cá nhân tham gia. Phương pháp này giúp nén giảm đáng kể khối lượng dữ liệu dư thừa nhưng vẫn bảo toàn trọn vẹn các đơn vị thông tin quan trọng (*information units*) phục vụ tìm kiếm.

Tối ưu hóa không gian tìm kiếm với Siêu dữ liệu (Metadata Filtering): Đề án đã lưu trữ trực tiếp các siêu dữ liệu (định danh người nói, mốc thời gian) dưới dạng *payload* vào cơ sở dữ liệu vector. Việc kết hợp song song giữa tìm kiếm ngữ nghĩa và bộ lọc metadata đã thu hẹp hiệu quả không gian truy xuất, loại bỏ hiện tượng nhầm lẫn thông tin giữa các diễn giả trong môi trường họp nhiều người.

6.1.1 Những hạn chế

Mặc dù đạt được những đột phá trong khâu xử lý ngôn ngữ, hệ thống vẫn tồn tại một số hạn chế nhất định. Thứ nhất, việc ứng dụng quy trình truy xuất đa bước lặp đi lặp lại và mô hình Cross-Encoder đòi hỏi chi phí tính toán cao, dẫn đến độ trễ phản hồi (*latency*) chưa thực sự tối ưu cho các tác vụ đòi hỏi thời gian thực. Thứ hai, quá trình đánh giá các mô-đun sinh văn bản hiện mới chỉ dừng lại ở phương pháp định tính thông qua kịch bản kiểm thử (*Case Studies*) do sự thiếu hụt của các bộ dữ liệu nhãn chuẩn (*Ground Truth*) đặc thù. Cuối cùng, đối với những trường hợp nhiễu ASR quá nghiêm trọng làm biến dạng hoàn toàn các chỉ dấu logic cốt lõi (ví dụ: nhầm lẫn phủ định thành khẳng định), cơ chế tự sửa lỗi của LLM vẫn có xác suất sinh ra những diễn giải sai lệch.

6.1.2 Định hướng phát triển

Từ những hạn chế trên, việc tối ưu hóa và tổ chức **metadata** cùng cấu trúc truy xuất sẽ là định hướng mở rộng then chốt của hệ thống trong tương lai:

Chỉ mục phân cấp (Hierarchical Indexing): Thay vì lưu trữ các đoạn hội thoại đồng cấp, xây dựng cấu trúc chỉ mục đồ thị theo chiều dọc (cuộc họp → chủ đề → người nói → đoạn hội thoại) nhằm tăng tốc độ truy vết cho các câu hỏi logic đan chéo.

Tích hợp Framework đánh giá RAG tự động: Sử dụng các công cụ đo lường chuyên biệt (như RAGAS hay TruLens) để lượng hóa các chỉ số hiệu năng cụ thể bao gồm độ phủ thông tin (*Context Recall*), độ chính xác ngữ cảnh (*Context Precision*) và tính xác thực (*Faithfulness*).

Tối ưu embedding cho truy xuất hỗn hợp: Sử dụng các mô hình embedding nhẹ cho các trường metadata để tăng tốc độ lọc sơ bộ, trong khi vẫn duy trì các embedding kích thước lớn, giàu ngữ nghĩa cho việc đánh giá độ tương đồng của nội dung hội thoại chính.

Nghiên cứu cơ chế Streaming RAG: Tối ưu hóa luồng dữ liệu để hệ thống có thể xử lý, phân mảnh và đánh chỉ mục song song ngay trong lúc cuộc họp đang diễn ra (*real-time indexing*), hướng tới việc xây dựng trợ lý ảo hỗ trợ trực tiếp (*live assistant*).

Bibliography

- [1] Vitalii Brydinskyi et al. “Comparison of Modern Deep Learning Models for Speaker Verification”. In: *Applied Sciences* 14.4 (2024). ISSN: 2076-3417. DOI: 10.3390/app14041329. URL: <https://www.mdpi.com/2076-3417/14/4/1329>.
- [2] Aryan Duhan et al. *Semantic Search and Recommendation Algorithm*. 2024. arXiv: 2412.06649 [cs.LG]. URL: <https://arxiv.org/abs/2412.06649>.
- [3] Yikun Han, Chunjiang Liu, and Pengfei Wang. “A Comprehensive Survey on Vector Database: Storage and Retrieval Technique, Challenge”. In: *arXiv preprint arXiv:2310.11703* (2023).
- [4] Ziwei Ji et al. “Survey of hallucination in natural language generation”. In: *ACM computing surveys* 55.12 (2023), pp. 1–38.
- [5] Vladimir Karpukhin et al. “Dense Passage Retrieval for Open-Domain Question Answering”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2020, pp. 6769–6781.
- [6] Patrick Lewis et al. “Retrieval-augmented generation for knowledge-intensive nlp tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474.
- [7] Pengfei Liu et al. “Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing”. In: *ACM Computing Surveys* 55.9 (2023), pp. 1–35.
- [8] Yu A Malkov and Dmitry A Yashunin. “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs”. In: *IEEE transactions on pattern analysis and machine intelligence* 42.4 (2018), pp. 824–836.

- [9] Rodrigo Nogueira and Kyunghyun Cho. “Passage Re-ranking with BERT”. In: *arXiv preprint arXiv:1901.04085* (2019).
- [10] Eduardo Pacheco et al. *SDBench: A Comprehensive Benchmark Suite for Speaker Diarization*. 2025. arXiv: 2507.16136 [cs.SD]. URL: <https://arxiv.org/abs/2507.16136>.
- [11] Tae Jin Park et al. “Auto-Tuning Spectral Clustering for Speaker Diarization Using Normalized Maximum Eigengap”. In: *IEEE Signal Processing Letters* 27 (2020), pp. 381–385. ISSN: 1558-2361. DOI: 10.1109/lsp.2019.2961071. URL: <http://dx.doi.org/10.1109/LSP.2019.2961071>.
- [12] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 2019, pp. 3982–3992.
- [13] Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 3rd. Prentice Hall, 2010.
- [14] Gerard Salton, Anita Wong, and Chung-Shu Yang. “A vector space model for automatic indexing”. In: *Communications of the ACM* 18.11 (1975), pp. 613–620.
- [15] Gemini Team et al. “Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context”. In: *arXiv preprint arXiv:2403.05530* (2024).
- [16] Nandan Thakur et al. “BEIR: A Heterogeneous Benchmark for Zero-shot Retrieval from Information Retrieval Models”. In: *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*. Vol. 1. 2021.
- [17] Harsh Trivedi et al. “Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions”. In: *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*. 2023, pp. 10014–10037.
- [18] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.

- [19] Jason Wei et al. “Chain-of-thought prompting elicits reasoning in large language models”. In: *Advances in neural information processing systems* 35 (2022), pp. 24824–24837.
- [20] Wenhan Xiong et al. “Answering complex open-domain questions with multi-hop dense retrieval”. In: *arXiv preprint arXiv:2009.12756* (2020).
- [21] Zhilin Yang et al. “HotpotQA: A dataset for diverse, explainable multi-hop question answering”. In: *Proceedings of the 2018 conference on empirical methods in natural language processing*. 2018, pp. 2369–2380.
- [22] Ming Zhong et al. “QMSum: A new benchmark for query-based multi-domain meeting summarization”. In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 2021, pp. 5905–5921.